UCEVA

INGENIERÍA DE SISTEMAS

PROYECTO INTEGRADOR - MANUAL TÉCNICO

Informe: Manual técnico del proyecto integrador (versión 3.0)

Semestre VI - Periodo 2026-2

Asignatura: Ingeniería del Software

Product Owner, Scrum Master, Development Team: UI/UX Designer, Frontend Developer,

Backend Developer, QA & Test Engineer, Documentation & DevOps Engineer

Octubre, 2026

Tabla de contenido

1. Objetivos 3
    1.1 Objetivos específicos 3
2. Alcance 4
3. Requerimientos técnicos 5
    3.1 Requerimientos mínimos de hardware 5
    3.2 Requerimientos mínimos de software 5
4. Herramientas utilizadas para el desarrollo 6
5. Instalación 7
6. Análisis 13
    6.1 Actores 13
    6.2 Historias de usuario 13
7. Diseño de la arquitectura 18
    7.1 Refinamiento de los Casos de Uso y Diagramas de Secuencia 18
    7.2 Control de versiones (Git Hub) 58
8. Desarrollo 59
9. Privilegios 70
    9.1 Roles y permisos 70
    9.2 Control de acceso en la API 71

1. Objetivos

Objetivo general

Se ha creado dicho documento con el propósito de mostrar cómo fue diseñado el sistema adaptable a dispositivos móviles y entornos web (UniLab), y al mismo tiempo dar referencias de cómo interactuar con el programa para que sea actualizado o al mismo tiempo se le dé un mantenimiento adecuado en caso de un fallo.

A grandes rasgos se diseñó con el propósito de guiar al programador que esté al frente de dicho sistema cómo se hizo, su arquitectura cliente-servidor (Node.js, PostgreSQL y Flutter/Frontend), su proceso de instalación, código fuente, servicios REST y control de versiones.

1.1 Objetivos específicos

- Guía de instalación y configuración de la aplicación y sus servicios backend.
- Mostrar el código fuente y la estructura de componentes (modelos, servicios, pantallas y base de datos) para una posible actualización del sistema en el futuro.
- Requisitos técnicos para la ejecución de dicha aplicación (incluyendo Node.js, PostgreSQL y dependencias del entorno multiplataforma).
- Manifestar evidencias del diseño del sistema antes de implementarlo, tales como casos de uso, diagramas de secuencia, modelo relacional y prototipos de interfaces (GUI).

2. Alcance

El sistema resuelve la falta de un canal ágil y centralizado para conocer la disponibilidad real de los espacios y reservarlos sin desplazarse ni depender de gestiones manuales. Se desarrolló con SCRUM en la asignatura Ingeniería del Software (semestre VI, periodo 2026-2, docente José Gabriel Pérez).

Incluye:

- Backend en Node.js/Express con autenticación (registro, login y recuperación de contraseña con OTP), laboratorios, horarios, reservas (crear, consultar, editar y cancelar) y asistencia.
- Base de datos PostgreSQL.
- App Flutter para web y Android con paneles de estudiante o egresado, administrador y encargado de laboratorio.

3. Requerimientos técnicos

3.1 Requerimientos mínimos de hardware

Ningún documento del repositorio ni del informe los define.

Equipo	Procesador	Memoria RAM	Disco
Desarrollo	Intel Core i5 / AMD Ryzen 5	8 GB	10 GB espacio libre (SSD recomendado)
Servidor	Dual-Core 2.0 GHz o superior	2 GB (4 GB si se incluye pgAdmin)	20 GB
Dispositivo móvil	Quad-Core 1.5 GHz	2 GB	100 MB libres

3.2 Requerimientos mínimos de software

Componente	Versión
Node.js y npm	18 o superior
PostgreSQL	16
Sistema operativo	Windows 10 u 11 (64 bits); también macOS o Linux de 64 bits
Chrome o Android Studio con emulador	Versión estable reciente de cada uno
JDK	17, porque el proyecto Android compila con Java 17
Git y Postman	Versión reciente

4. Herramientas utilizadas para el desarrollo

Categoría	Herramienta	Uso
App móvil y web	Flutter y Dart	Interfaz
Backend	Node.js y Express	API REST
Base de datos	PostgreSQL 16	Administración de los registros almacenados
Pruebas	Postman y `flutter_test`	Colecciones de la API y pruebas de la app
Control de versiones	Git y GitHub	GitFlow con Pull Requests
Editores	Visual Studio Code y Android Studio	Código
Modelado y prototipo	Figma y prototipo en HTML	Diseño de las GUIs
Comunicación	Figma y prototipo HTML WhatsApp	Coordinación de los equipos

5. Instalación

Antes de comenzar, asegúrese de tener instalado el siguiente software en su equipo:

- Git: Para clonar el repositorio.
- Node.js: (v18 o superior) Para el entorno del Backend.
- PostgreSQL: Gestor de base de datos relacional.
- Flutter SDK: (Canal estable, SDK de Dart 3.x) Para el Frontend.
- Navegador Web: Google Chrome o Microsoft Edge (para ejecutar el frontend en modo web).

Configuración del Backend (API)

1. Clonar el repositorio

El primer paso para instalar el proyecto es descargar el código fuente desde GitHub. Abre tu terminal (Git Bash, PowerShell o la terminal de tu editor de código) en la carpeta donde desees guardar el proyecto y ejecuta los siguientes comandos:

```
git clone https://github.com/robinsonperez02/PI_UNILAB.git
cd PI_UNILAB
git checkout develop
```

Explicación de los comandos:

1. git clone https://github.com/robinsonperez02/PI_UNILAB.git: Descarga una copia exacta del repositorio remoto a tu computadora. Se creará una carpeta llamada PI_UNILAB.
2. cd PI_UNILAB: Nos permite ingresar (moverse) a la carpeta que se acaba de crear.
3. git checkout develop: Cambia la rama actual del repositorio a la rama develop (desarrollo).

2. Instalación de dependencias y configuración inicial del Backend

Una vez clonado el repositorio, abre una terminal en la raíz del proyecto y ejecuta los siguientes comandos en orden para preparar el servidor:

Backend

```
cd backend
npm install
cp .env.example .env   # configurar variables de entorno (BD, JWT, etc.)
npm run dev
```

Explicación de los comandos:

1. cd backend: Nos posiciona dentro de la carpeta que contiene el código fuente del servidor.
2. npm install: Descarga e instala todas las librerías y dependencias de Node.js necesarias para que el backend funcione.
3. cp .env.example .env: Copia el archivo de plantilla .env.example y crea un archivo real llamado .env.

3. Configuración inicial del Frontend

Una vez clonado el repositorio, abre una nueva terminal (deja la del backend corriendo en la anterior) y dirígete a la carpeta de la aplicación móvil/web. Ejecuta los siguientes comandos:

```
cd app
flutter pub get        # o: npm install (React Native)
flutter run             # o: npx react-native run-android
```

Explicación de los comandos:

1. cd app: Ingresa a la carpeta donde se encuentra el código fuente del Frontend (en este proyecto se llama app).
2. flutter pub get: Descarga e instala todas las dependencias y paquetes de Flutter necesarios para que la aplicación funcione. *(Nota: Si el proyecto estuviera hecho en React Native, se usaría* npm install *como indica el comentario, pero para este proyecto usamos Flutter).*
3. flutter run: Compila y ejecuta la aplicación.

3.1. Preparación del entorno Backend

El primer paso es asegurarte de estar en la rama correcta. Sigue estos pasos en tu terminal:

```
backend
```

Explicación de los pasos:

1. Cambiar a la rama backend: Si vienes de la sección anterior (Clonar repositorio), asegúrate de ejecutar git checkout backend para traer el código del servidor.

```
npm install
```

2. Instalar dependencias: Ejecuta el comando npm install.

¿Qué hace este comando? Lee el archivo package.json del proyecto y descarga automáticamente todas las librerías y paquetes de Node.js que la API necesita para funcionar (como Express, bases de datos, seguridad, etc.). Es un paso obligatorio antes de intentar iniciar el servidor.

3.2. Configuración de la Base de Datos

El backend necesita una base de datos para guardar la información de las reservas. Sigue estos pasos para prepararla:

`bd_saici` en PostgreSQL y ejecutar el script `BD_Saici.sql`.

¿Cómo hacerlo paso a paso?

1. Abrir PostgreSQL: Abre tu gestor de bases de datos (como pgAdmin, DBeaver o la consola de PostgreSQL).
2. Crear la base de datos: Crea una nueva base de datos vacía y asígnale exactamente el nombre bd_saici.
    -

1. Ejecutar el script: Busca el archivo BD_Saici.sql (generalmente ubicado en la carpeta raíz del backend). Ábrelo o impórtalo desde tu gestor de base de datos y ejecútalo.
    - *Esto creará automáticamente todas las tablas, columnas y relaciones necesarias para que el sistema funcione.*

3.3. Configuración de las Variables de Entorno

El backend necesita conocer datos sensibles (como tus contraseñas de la base de datos) para poder funcionar. Estos datos no se suben a GitHub por seguridad, por lo que debes crearlos tú mismo.

Crea un archivo llamado exactamente .env en la raíz de la carpeta del backend y pega el siguiente contenido:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=bd_saici
DB_USER=tu_usuario
DB_PASSWORD=tu_contraseña
JWT_SECRET=una_clave_larga_y_secreta
EMAIL_USER=correo_que_envia_los_codigos@gmail.com
EMAIL_PASS=contraseña_de_aplicacion_de_gmail
```

Explicación de cada variable

- DB_HOST y DB_PORT: La dirección y el puerto donde está tu base de datos. Por defecto en PostgreSQL es localhost y 5432.
- DB_NAME: El nombre de la base de datos que creaste en el paso anterior (bd_saici).
- DB_USER y DB_PASSWORD: Tu usuario y contraseña de PostgreSQL (ej. usuario: postgres, contraseña: admin).
- JWT_SECRET: Una palabra o frase larga y secreta que se usa para encriptar los inicios de sesión. Puede ser cualquier texto inventado (ej. mi_clave_super_secreta_2024).
- EMAIL_USER y EMAIL_PASS: El correo desde el cual el sistema enviará los códigos de recuperación.

3.4. Iniciar el servidor Backend

Con las dependencias instaladas y las variables de entorno configuradas, ya estás listo para encender el servidor. En tu terminal (asegurándote de estar dentro de la carpeta backend), ejecuta uno de los siguientes comandos según lo que necesites hacer:

```
npm run dev     # desarrollo, se reinicia solo
npm start       # ejecución normal
```

Explicación de los comandos:

- npm run dev: Inicia el servidor en modo desarrollo. Este es el comando recomendado si vas a modificar el código o revisar errores, ya que el servidor se reinicia automáticamente cada vez que guardas un cambio en tus archivos.
- npm start: Inicia el servidor en modo normal (o producción). Se usa cuando el código ya está terminado y solo quieres ejecutarlo de forma estable, sin que se esté reiniciando constantemente.

3.5. Verificación del Backend

Para confirmar que el servidor se ha iniciado correctamente y está escuchando peticiones, realiza la siguiente prueba:

Comprobar que funciona: abrir `http://localhost:3000/`. Debe responder `{"mensaje": "API de reservas funcionando correctamente"}`.

---

1. Abre tu navegador web (Chrome, Edge, etc.).
2. Ingresa a la siguiente dirección: http://localhost:3000/
3. Si todo está correcto, la pantalla del navegador mostrará un mensaje en formato JSON similar a este:
   {"mensaje": "API de reservas funcionando correctamente"}.

Configuración del Frontend (API)

4.1. Requisitos previos para el Frontend

Antes de intentar ejecutar la interfaz gráfica de la aplicación, es necesario asegurarse de que el entorno de desarrollo esté listo. Para este proyecto, necesitas:

- Flutter: (Canal stable, SDK de Dart 3.x). Asegúrate de tenerlo instalado y configurado en las variables de entorno de tu sistema.
- Navegador Web: Google Chrome o Microsoft Edge (ya que ejecutaremos la aplicación en modo web).

Verificación de la instalación:

Abre una nueva terminal y ejecuta el siguiente comando :

```
flutter doctor
```

- ¿Qué hace este comando? Realiza un diagnóstico completo de tu entorno. Deberías ver una lista con "checks" (palomitas verdes) indicando que Flutter, Dart y el soporte para desarrollo web están correctamente instalados. Si ves algún error en rojo, deberás solucionarlo antes de continuar.

4.2. Ejecutar la aplicación en local

Una vez verificado el entorno, abre una terminal en la carpeta raíz del frontend (app) y ejecuta los siguientes comandos:

```
flutter pub get
flutter run -d chrome
```

Explicación de los comandos:

1. flutter pub get: Descarga e instala todas las librerías y dependencias necesarias para que la aplicación funcione.
2. flutter run -d chrome: Compila el código y abre automáticamente la aplicación en una ventana de Google Chrome. *(El parámetro* -d chrome *le indica a Flutter que queremos ejecutarlo en el navegador web).*

4.3. Pruebas y Análisis de Calidad

Antes de desplegar la aplicación o entregar el proyecto, es una buena práctica asegurarse de que el código no tenga errores de sintaxis ni fallos en las pruebas unitarias. Para ello, ejecuta los siguientes comandos en la terminal del frontend:

```
flutter analyze
flutter test
```

- flutter analyze: Realiza un análisis estático del código. Revisa todos los archivos en busca de errores de sintaxis, malas prácticas o advertencias (warnings). Si todo está bien, el resultado debe indicar "No issues found!" (¡No se encontraron problemas!).
- flutter test: Ejecuta todas las pruebas unitarias y de widgets que se hayan programado en el proyecto. Te mostrará un reporte de cuántas pruebas pasaron (All tests passed!) y cuántas fallaron.

4.4. Compilación para Producción

Cuando la aplicación esté lista para ser publicada en un servidor real, no se debe usar el modo de desarrollo (flutter run). En su lugar, debes generar una versión optimizada y comprimida para el usuario final.

```
flutter build web --release
```

Explicación del comando:

- flutter build web --release: Compila todo el código de Flutter a HTML, CSS y JavaScript puro. El parámetro --release optimiza el código para que cargue más rápido y ocupe menos espacio.

5. Notas y Advertencias Finales

Para concluir, ten en cuenta las siguientes consideraciones importantes sobre la estructura del proyecto y el control de versiones:

- `android/local.properties` **no** se versiona: se genera automáticamente con `flutter pub get` o desde tu IDE según la ruta de tu SDK de Android.
- El backend, las migraciones SQL y los archivos de Docker **no** forman parte de esta rama.

6. Análisis

El informe desarrolla esta etapa en sus secciones: historias de usuario, casos de uso con diagramas de secuencia y prototipos de interfaces.

6.1 Actores

Actor	Descripción
Usuario cliente (egresado o estudiante de semillero)	Usa la app móvil: consulta disponibilidad, reserva, cancela, ve su historial y su perfil
Administrador	Usa el panel administrativo: gestiona reservas, espacios, horarios, usuarios y reportes
Encargado de laboratorio	Consulta las sesiones programadas y la información de los laboratorios
Backend / API REST	Actor autónomo: valida reglas de negocio y detecta conflictos de horario
Base de datos	Actor de almacenamiento: garantiza la integridad de los datos

6.2 Historias de usuario

ID	**Historia de Usuario**	**Descripción breve**	**Caso(s) de Uso Relacionados**	**GUI Relacionada**
HU-F01	Iniciar sesión	Cada vez que un usuario desee ingresar al sistema, deberá digitar su nombre de usuario y contraseña. Internamente el sistema validará la información para permitir o rechazar el acceso.	CU_01	Pantalla Inicio de Sesión. GUI-01

ID	Historia de Usuario	Descripción breve	Caso(s) de Uso Relacionados	GUI Relacionada
HU-F02	Cerrar sesión	El sistema permitirá al usuario finalizar su sesión de forma segura, terminando todos los procesos abiertos.	CU_03	Pantalla de Inicio / Dashboard
HU-F03	Consultar disponibilidad	Cuando el usuario desee realizar una consulta, el sistema le permitirá ver la información de la disponibilidad de los espacios para reservar.	CU_02	Pantalla de Inicio / Dashboard
HU-F04	Consultar información del laboratorio	Cuando el usuario desee conocer la información básica de cada laboratorio, el sistema le mostrará: Ubicación del laboratorio Nombre del laboratorio, disponibilidad de recursos (energía, conectividad) Horario disponible	CU_02	Pantalla de Inicio / Dashboard
HU-F05	Reservar un espacio por fecha y horario	Cuando un usuario desee gestionar las reservas de un laboratorio, el sistema deberá permitirle seleccionar un laboratorio, consultar las reservas asociadas al mismo. Para realizar una nueva reserva debe definir la fecha, hora de inicio y hora de finalización, con el fin de conocer y gestionar la disponibilidad y utilización del espacio.	CU_02	Asistente de Reservas
HU-F07	Evitar conflictos de horarios	El sistema verificará la información de la nueva reserva. Revisar el espacio, para que no se encuentre ya reservado. Si se encuentra reservado, el sistema deberá tomarlo como un solapamiento e impedir la reserva al usuario. El sistema le notificará al usuario que el laboratorio no está disponible en este horario, además le mostrará horarios disponibles. El sistema verifica que un laboratorio no esté ya reservado en la misma fecha y horario, rechazando la nueva reserva y avisando al usuario si existe solapamiento.	CU_02	Asistente de Reservas
HU-F09	Cancelar una reserva	Permite al egresado cancelar una reserva, cambiando su estado a "cancelada" y liberando el laboratorio para que otros usuarios puedan reservarlo	CU_02	Mis Reservas e Historial
HU-F10	Consultar mis reservas	El Sistema permite al egresado consultar todas sus reservas activas, mostrando la información	CU_02	Mis Reservas e Historial

ID	Historia de Usuario	Descripción breve	Caso(s) de Uso Relacionados	GUI Relacionada
		del laboratorio, la fecha, el horario y el estado actual.		
HU-F11	Consultar y filtrar historial de reservas	Cuando el egresado desee consultar su historial, el sistema mostrará todas sus reservas pasadas, finalizadas o canceladas, ordenadas por fecha descendente, incluyendo información como el laboratorio, la fecha, el horario y el estado. Además, podrá filtrar el historial por fecha o estado para encontrar rápidamente una reserva específica.	CU_02	Mis Reservas e Historial
HU-F12	Consultar información personal	El egresado puede acceder a los datos de su propia cuenta (nombre, correo, etc.) para verificarlos, asegurando que solo vea su información y no la de otros usuarios.	CU_07	Vista Perfil
HU-F13	Gestionar espacios	Como administrador del sistema, quiero crear, editar y activar/desactivar los laboratorios disponibles para reserva, para mantener actualizada la información que ven los egresados al reservar	CU_03	Panel Administrativo
HU-F14	Gestionar reservas desde administración	Permite al administrador consultar y modificar las reservas realizadas por los usuarios, mostrando información como el laboratorio reservado, usuario, fecha, hora y estado, con el fin de facilitar el control y la gestión de los espacios.	CU_03	Panel Administrativo
HU-F15	Generar estadísticas de uso	El sistema permitirá al usuario consultar información y generar estadísticas sobre la utilización de los laboratorios, como el número de reservas realizadas, la frecuencia de utilización y los espacios con mayor o menor uso, para apoyar la planeación de los recursos y la toma de decisiones administrativas.	CU_06	Panel Administrativo
HU-F16	Consultar porcentaje de ocupación	Permite al usuario (administrador) consultar el porcentaje de ocupación de uno o varios laboratorios, calculado como la razón entre las horas reservadas y las horas totales disponibles dentro de un rango de fechas seleccionado por el usuario (día, semana, mes). El sistema debe permitir filtrar la consulta por laboratorio específico o por todos los laboratorios y mostrar el resultado en formato numérico	CU_06	Panel Administrativo

ID	Historia de Usuario	Descripción breve	Caso(s) de Uso Relacionados	GUI Relacionada
		(porcentaje) actualizándose automáticamente según las reservas y cancelaciones registradas.		
HU-F17	Actualizar disponibilidad después de una reserva	El sistema debe recalcular el estado del laboratorio (disponible/ocupado) en tiempo real, sin intervención manual, cada vez que ocurra: se confirma una reserva, se actualiza una reserva, o termina el horario de una reserva activa. Esto debe verse reflejado tanto en la consulta de disponibilidad como en la información del laboratorio.	CU_02	Pantalla de Inicio / Dashboard
HU-F18	Consumir API REST	Cuando la aplicación móvil necesite intercambiar información con el backend, el sistema permitirá consumir servicios REST para enviar y recibir datos. Estos servicios facilitarán la comunicación entre la aplicación móvil, el backend y la base de datos, permitiendo realizar las operaciones necesarias de consulta, registro, actualización y gestión de la información.	CU_05	No aplica
HU-F19	Validar reglas de reserva	El sistema verificará que la solicitud de reserva cumpla con las reglas establecidas, como la disponibilidad del laboratorio, la fecha y hora seleccionadas y que no exista una reserva previa para el mismo espacio. Si alguna regla no se cumple, el sistema rechazará la reserva e informará al usuario el motivo	CU_02	Asistente de Reservas
HU-F21	Recuperar acceso a la cuenta	Cuando un usuario olvide su contraseña, el sistema le permitirá solicitar la recuperación de acceso mediante su correo electrónico registrado, enviando un enlace o código temporal.	CU_01	Pantalla Recuperación Cuenta. GUI-03
HU-F22	Consultar horarios disponibles	Cuando el usuario desee conocer la disponibilidad de un laboratorio, el sistema le permitirá seleccionar una fecha y un espacio (laboratorio); internamente el sistema consultará las reservas existentes y mostrará únicamente los horarios disponibles para esa combinación de fecha y espacio	CU_02	Asistente de Reservas

ID	Historia de Usuario	Descripción breve	Caso(s) de Uso Relacionados	GUI Relacionada
HU-F25	Consultar estado de los laboratorios	Cuando el usuario desee consultar el estado de los laboratorios, el sistema le permitirá visualizar cuáles se encuentran activos o deshabilitados, con el fin de identificar los espacios disponibles para realizar nuevas reservas.	CU_02	Pantalla de Inicio / Dashboard
HU-F26	Registrar horarios de laboratorio	El sistema deberá permitir al administrador registrar, modificar, consultar y eliminar los horarios de disponibilidad de los laboratorios, con el fin de establecer los días y períodos en los que los egresados podrán realizar reservas.	CU_03	Panel Administrativo
HU-F27	Modificar horario de laboratorio	El sistema debe permitir al administrador cancelar la disponibilidad del horario de uso del laboratorio ante cualquier evento inesperado, requiriendo ingresar el nuevo horario (o marcar la cancelación) junto con la justificación obligatoria del cambio.	CU_03	Panel Administrativo
HU-F28	Consultar usuarios del sistema	Debe permitir al usuario consultar otros usuarios registrados en el sistema para verificar quienes tienen acceso a la plataforma y tener un mejor control ante las o la sala que se necesite en ese momento.	CU_07	Panel Administrativo
HU-F30	Consultar reportes de utilización	El sistema deberá permitir a los usuarios autorizados consultar y generar reportes sobre el uso de los laboratorios y las reservas realizadas. Estos reportes deberán presentar la información de manera clara y organizada, de modo que sea fácil conocer cómo se están utilizando los espacios y cuáles son sus niveles de ocupación. Para realizar la consulta, el usuario podrá seleccionar un rango de fechas y aplicar filtros como el laboratorio, usuario y estado de la reserva. Con base en los criterios seleccionados, el sistema mostrará la información correspondiente al período consultado. El reporte deberá incluir información como: ● Nombre e identificación del laboratorio. ● Fecha de la reserva.	CU_04	Panel Administrativo

ID	Historia de Usuario	Descripción breve	Caso(s) de Uso Relacionados	GUI Relacionada
		• Hora de inicio y finalización. • Usuario que realizó la reserva. • Estado de la reserva. • Tiempo total reservado. • Número de reservas realizadas. • Número de reservas canceladas o que no fueron utilizadas. • Porcentaje de ocupación del laboratorio. La información podrá visualizarse mediante tablas y gráficos, permitiendo comparar fácilmente el uso de los diferentes laboratorios e identificar cuáles tienen mayor o menor ocupación. Los resultados deberán actualizarse cada vez que el usuario modifique los filtros de consulta. Además, el acceso a estos reportes estará limitado a los usuarios que tengan los permisos necesarios para consultarlos.		
HU-F31	Gestión y asignación de rol de usuario	El sistema debe clasificar a los usuarios en dos roles principales: Egresado o Estudiante del laboratorio. Cada rol determinará los permisos de acceso y el flujo de uso dentro de la plataforma (acceso mediante reserva previa para egresados o acceso directo/asistencia libre para estudiantes del laboratorio).	CU_06	Vista Perfil / Registro de Usuario GUI-02
HU-F32	Acceso y asistencia directa de estudiantes del laboratorio	Cuando un usuario con el rol Estudiante del laboratorio ingrese a las instalaciones, el sistema le permitirá acceder a los espacios físicos y recursos del laboratorio sin requerir la consulta de disponibilidad ni la creación previa de una reserva, ya que cuenta con autorización permanente como miembro del laboratorio.	CU_07	Vista Inicio / Panel Estudiante
HU-F33	Reporte y registro de horas de asistencia	El sistema deberá permitir al Estudiante del laboratorio registrar y reportar las horas en las que asiste al laboratorio, indicando fecha, hora de entrada y hora de salida (o mediante marcación en tiempo real). Esta información quedará almacenada en la base de datos para el	CU_07	Vista Registro de Asistencia / Mis Reservas e Historial

ID	Historia de Usuario	Descripción breve	Caso(s) de Uso Relacionados	GUI Relacionada
		seguimiento de sus horas y control de permanencia.		

7. Diseño de la arquitectura

7.1 Refinamiento de los Casos de Uso y Diagramas de Secuencia

Figura 1. *Caso de Uso Registrar Usuario.*

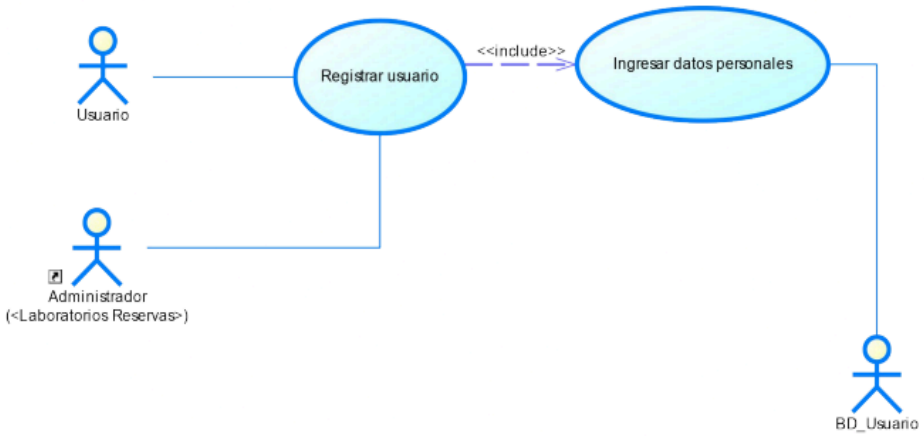


Tabla 3. *Especificación del Caso de Uso CU-00 (Registrar Usuario).*

Especificación	Detalle
ID	CU-00
Nombre	Registrar usuario
Actor(es)	Usuario, Administrador, BD_Usuario
Objetivo	Permitir la creación de una nueva cuenta en el sistema, asegurando la captura correcta y segura de la información personal del egresado.
Precondiciones	El usuario no debe poseer una cuenta activa previamente vinculada a su correo electrónico institucional o número de teléfono.
Postcondiciones	El perfil del usuario queda registrado de forma segura y persistido en la base de datos (BD_Usuario), asignándole el rol correspondiente para acceder a la plataforma.

Flujo Principal	1. El Usuario (o el Administrador) inicia el proceso para crear una cuenta. 2. El sistema despliega el formulario de registro. 3. El usuario provee su información (esto **incluye** el proceso de *Ingresar datos personales*). 4. El sistema valida que los campos cumplan con los formatos requeridos. 5. El sistema procesa la solicitud y envía la información a la **BD_Usuario** para su almacenamiento. 6. El sistema muestra un mensaje de confirmación de registro exitoso en pantalla.
Flujos Alternativos	• **Excepción 1 (Datos duplicados):** El sistema detecta que el correo o el teléfono ya existen en la base de datos e informa al usuario que ya está registrado. • **Excepción 2 (Error de validación):** Las contraseñas no coinciden o faltan campos obligatorios; el sistema solicita corregir la información.
Reglas de Negocio	• Todos los datos personales solicitados en el formulario son de diligenciamiento obligatorio. • El número de teléfono debe contener exactamente 10 dígitos. • Las contraseñas se deben almacenar de forma encriptada en la base de datos y no en texto plano.
Datos Involucrados	• Nombres y Apellidos. • Correo electrónico. • Teléfono. • Contraseña (cifrada).
GUI Asociada	Pantalla de "Crear cuenta de egresado" (Formulario de Registro).

Figura 1.1 *Diagrama de Secuencia Registrar Usuario*

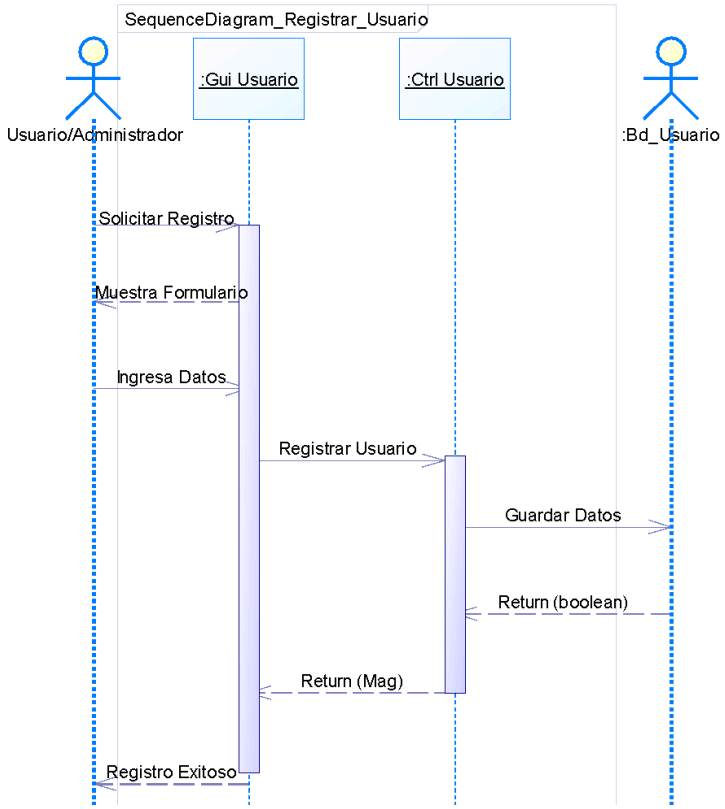


Tabla 3.1. *Especificación del Diagrama de Frecuencia (Registrar Usuario).*

Especificación	**Detalle**
Caso de uso	Registrar usuario
ID	CU-00
Actores	Usuario / Administrador, :Bd_Usuario
Objetos	:Gui_Usuario, :Ctrl_Usuario
Precondición	El usuario o el administrador accede a la opción de registro.
Postcondición	Los datos personales del usuario quedan almacenados en BD_Usuario.
Descripción del flujo	El usuario o el administrador solicita registrar un usuario. La interfaz (:Gui_Usuario) muestra el formulario y recibe los datos personales; el controlador (:Ctrl_Usuario) los procesa y los guarda en :Bd_Usuario. Al confirmarse el guardado, la interfaz informa que el registro fue exitoso.

Figura 2. *Caso de Uso Iniciar Sesión.*

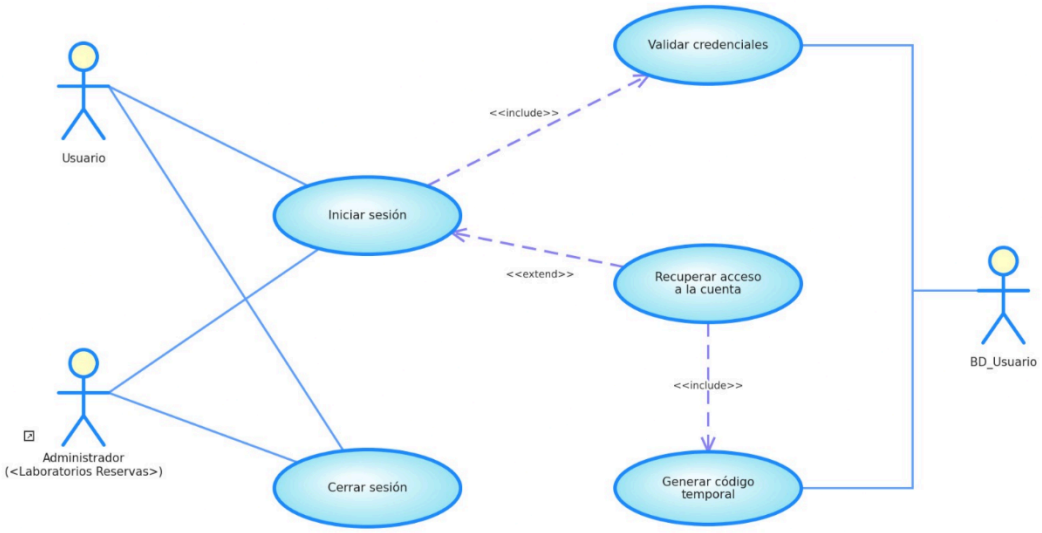


Tabla 4. *Especificación del Caso de Uso CU-01 (Iniciar Sesión).*

Especificación	**Detalle**
ID	CU-01
Nombre	Iniciar Sesión
Actor(es)	Usuario, Administrador
Objetivo	Permitir que el usuario o administrador acceda al sistema mediante sus credenciales.
Precondiciones	El usuario debe estar registrado en el sistema, el usuario debe encontrarse en la pantalla de inicio de sesión y debe existir conexión con la base de datos de usuario.
Postcondiciones	El usuario queda autenticado, el sistema permite acceder a las funcionalidades correspondientes a su rol y se inicia una sesión activa.
Flujo Principal	1. El usuario ingresa a la pantalla de inicio de sesión. 2. El sistema solicita usuario/correo y contraseña. 3. El usuario introduce sus credenciales.

	4. El sistema recibe los datos ingresados. 5. El sistema verifica que las credenciales sean correctas. 6. El sistema identifica el rol del usuario. 7. El sistema permite el acceso. 8. El sistema muestra la pantalla principal correspondiente al rol.
Flujos Alternativos	• **Credenciales incorrectas:** El sistema informa que el usuario o contraseña son incorrectos. • **Campos vacíos:** El sistema solicita completar los campos obligatorios. • **Usuario no registrado:** El sistema informa que no existe una cuenta asociada. • **Error de conexión:** El sistema informa que no fue posible validar las credenciales.
Reglas de Negocio	• Solo usuarios registrados pueden acceder. • Cada usuario debe ingresar credenciales válidas. • Las funcionalidades disponibles dependen del rol asignado.
Datos Involucrados	• Usuario/correo. • Contraseña. • Identificador del usuario. • Rol. • Estado de la cuenta.
GUI Asociada	• Pantalla de Inicio de sesión. • Campos de usuario/correo y contraseña. • Botón Iniciar sesión.

Tabla 5. *Especificación del Caso de Uso CU-01 (Validar Credenciales).*

Especificación	Detalle
ID	CU-01
Nombre	Validar credenciales
Actor(es)	Usuario, Administrador, BD_Usuario
Objetivo	Verificar que las credenciales proporcionadas correspondan a un usuario registrado.
Precondiciones	El usuario debe haber ingresado usuario/correo y contraseña. La base de datos de usuarios debe estar disponible.
Postcondiciones	Las credenciales quedan validadas correctamente o son rechazadas. Si son correctas, el sistema permite continuar con el inicio de sesión.
Flujo Principal	1. El usuario ingresa sus credenciales. 2. El sistema recibe el usuario/correo y contraseña. 3. La base de datos devuelve la información correspondiente. 4. El sistema compara las credenciales ingresadas con las almacenadas. 5. El sistema verifica el estado de la cuenta. 6. El sistema confirma que las credenciales son válidas. 7. El sistema permite continuar con CU-01: Iniciar sesión.
Flujos Alternativos	• **Contraseña incorrecta:** El sistema rechaza el acceso. • **Usuario inexistente:** El sistema informa que las credenciales no son válidas. • **Cuenta inactiva:** El sistema impide el acceso. • **Base de datos no disponible:** El sistema informa que no puede realizar la validación.
Reglas de Negocio	• Las credenciales deben coincidir con la información registrada. • Una cuenta inactiva no puede iniciar sesión. • La contraseña no debe mostrarse directamente en pantalla.

Datos Involucrados	• Usuario/correo. • Contraseña. • ID de usuario. • Rol. • Estado de la cuenta.
GUI Asociada	• Pantalla de Inicio de Sesión. • Mensajes de validación de credenciales.

Tabla 6. *Especificación del Caso de Uso CU-01 (Recuperar acceso a la cuenta).*

Especificación	**Detalle**
ID	CU-01
Nombre	Recuperar acceso a la cuenta
Actor(es)	Usuario, Administrador
Objetivo	Permitir al usuario recuperar el acceso a su cuenta cuando no puede iniciar sesión.
Precondiciones	El usuario debe tener una cuenta registrada. El usuario debe encontrarse en la pantalla de inicio de sesión. Debe existir un mecanismo para verificar la identidad del usuario.
Postcondiciones	El usuario recibe un código temporal. El usuario puede continuar con el proceso de recuperación. Si completa correctamente el proceso, puede recuperar el acceso a su cuenta.
Flujo Principal	1. El usuario selecciona la opción Recuperar acceso a la cuenta. 2. El sistema solicita los datos necesarios para identificar la cuenta. 3. El usuario proporciona la información solicitada. 4. El sistema verifica que la cuenta exista.

	5. El sistema genera un código temporal. 6. El sistema envía o muestra el mecanismo para utilizar el código. 7. El usuario ingresa el código recibido. 8. El sistema valida el código. 9. El sistema permite continuar con la recuperación de la cuenta.
Flujos Alternativos	• **Cuenta no encontrada:** El sistema informa que no existe una cuenta asociada. • **Código incorrecto:** El sistema informa que el código ingresado no es válido. • **Código expirado:** El sistema solicita generar un nuevo código. • **Datos incorrectos:** El sistema solicita verificar nuevamente la información ingresada.
Reglas de Negocio	• El código temporal debe tener un tiempo limitado de validez. • El código sólo puede utilizarse durante el período establecido. • La recuperación debe realizarse únicamente para cuentas existentes.
Datos Involucrados	• Usuario/correo. • Identificador de cuenta. • Código temporal. • Fecha y hora de generación. • Fecha y hora de expiración.
GUI Asociada	• Pantalla Recuperar acceso. • Campo de usuario/correo. • Campo para ingresar código temporal. • Botón Continuar.

Figura 2.1 *Diagrama de Secuencia Iniciar Sesión*

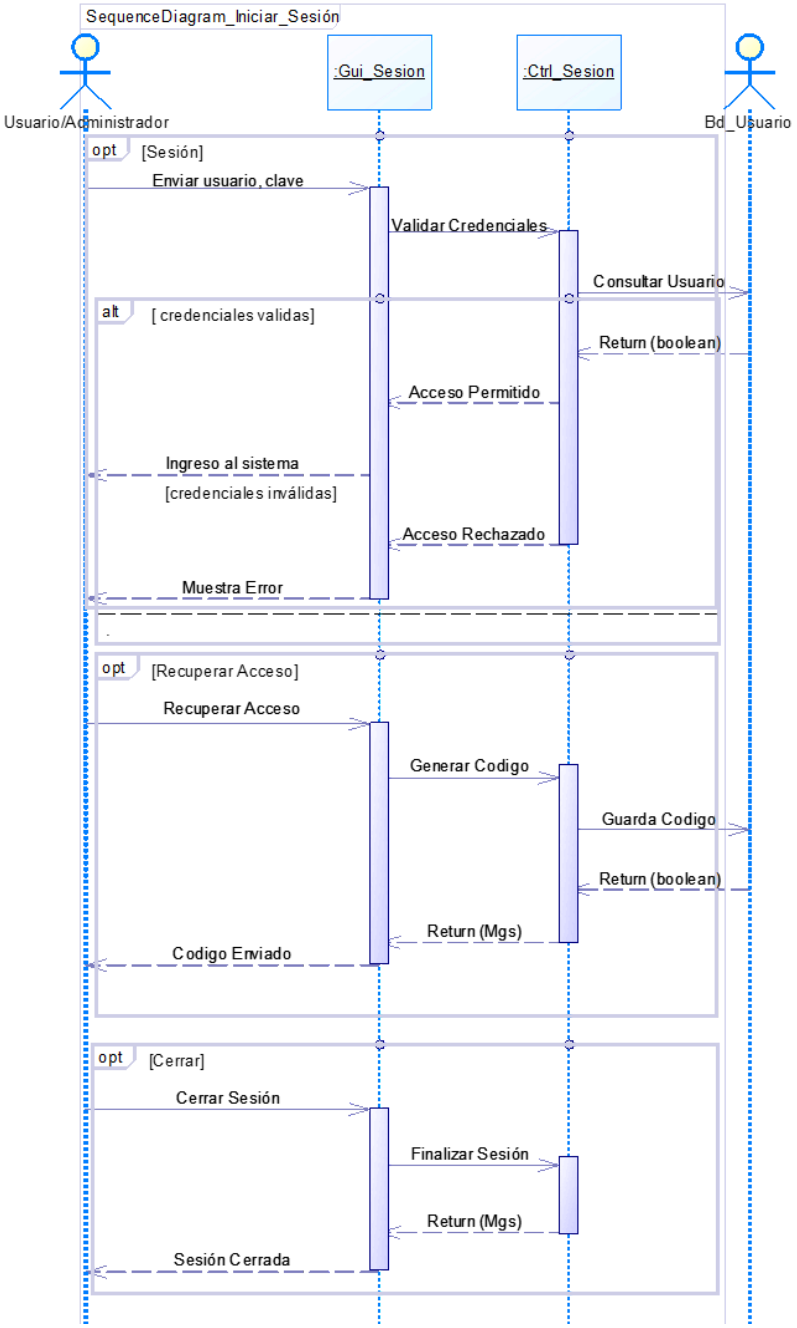


Tabla 6.1. *Especificación del Diagrama de Secuencia (Iniciar Sesión).*

Especificación	**Detalle**
Caso de uso	Iniciar sesión
ID	CU-01

Especificación	Detalle
Actores	Usuario / Administrador, Bd_Usuario
Objetos	:Gui_Sesion, :Ctrl_Sesion
Precondición	El usuario o el administrador está registrado en el sistema.
Postcondición	Se permite el acceso al sistema o se rechaza si las credenciales no son válidas; la sesión puede cerrarse al finalizar.
Descripción del flujo	El usuario o el administrador envía su nombre de usuario y contraseña. El controlador valida las credenciales consultando Bd_Usuario y, según el resultado, el sistema permite el ingreso (mensajes 5 y 6) o lo rechaza mostrando un error (mensajes 7 y 8). Si el acceso falla, el usuario puede recuperar el acceso: el sistema genera un código temporal, lo guarda en Bd_Usuario y se lo envía al usuario (mensajes 9 al 14). Finalmente, el usuario puede cerrar sesión (mensajes 15 al 18).

Figura 3. *Caso de Uso Gestionar Reserva.*

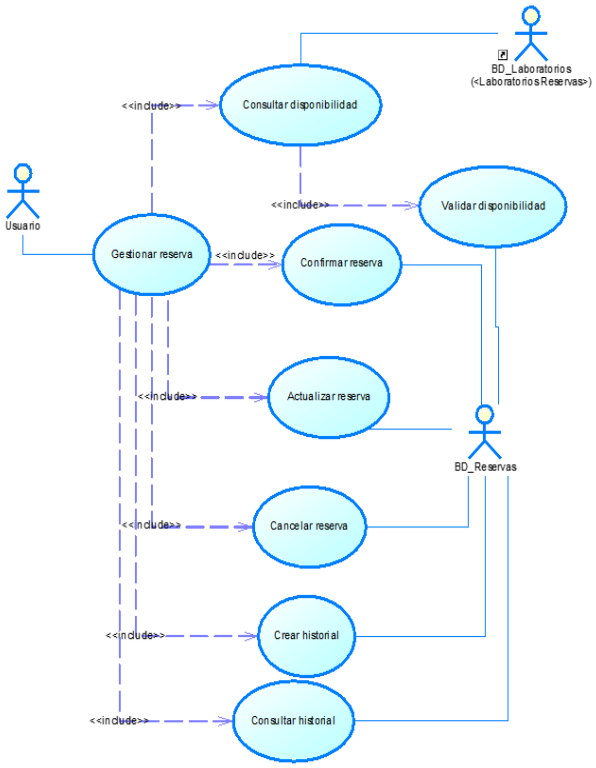


Tabla 7. *Especificación del Caso de Uso CU-2 (Gestionar Reserva).*

Especificación	Detalle
ID	CU-02
Nombre	Gestionar reserva
Actor(es)	Usuario, BD_Laboratorios, BD_Reservas
Objetivo	Permitir al usuario realizar el proceso integral de gestión de reservas de laboratorios.
Precondiciones	El usuario debe estar autenticado en el sistema.
Postcondiciones	La reserva y sus movimientos quedan registrados y procesados en la base de datos.
Flujo Principal	1. El usuario inicia el proceso de gestión de reserva. 2. El sistema ejecuta la consulta de disponibilidad de laboratorios (incluye Consultar disponibilidad). 3. El sistema valida la disponibilidad con la base de datos (incluye Validar disponibilidad). 4. El usuario confirma los datos de la reserva (incluye Confirmar reserva). 5. El sistema registra/actualiza la información en la base de datos (incluye Actualizar reserva y Crear historial). 6. El sistema muestra la confirmación en pantalla.
Flujos Alternativos	• **Excepción 1:** El laboratorio seleccionado no cuenta con disponibilidad en la fecha y hora seleccionadas. • **Excepción 2:** Error de conexión con BD_Laboratorios o BD_Reservas.
Reglas de Negocio	Solo se permite reservar si la disponibilidad es validada satisfactoriamente en BD_Laboratorios.
Datos Involucrados	• D_Usuario. • ID_Laboratorio. • Fecha. • Hora_Inicio.

	• Hora_Fin.Estado_Reserva.
GUI Asociada	Pantalla Principal de Gestión de Reservas / Formulario de Reserva.

Tabla 8. *Especificación del Caso de Uso CU-02 (Consultar Disponibilidad).*

Especificación	**Detalle**
ID	CU-02
Nombre	Consultar disponibilidad
Actor(es)	Usuario, BD_Laboratorios
Objetivo	Verificar qué laboratorios y horarios están disponibles para su reserva.
Precondiciones	El usuario debe ingresar los criterios de búsqueda (laboratorio, fecha o rango de horas).
Postcondiciones	El sistema despliega la lista de laboratorios y horarios disponibles.
Flujo Principal	1. El usuario/sistema solicita la lista de laboratorios y sus horarios. 2. El sistema consulta la base de datos BD_Laboratorios. 3. El sistema invoca el caso de uso "Validar disponibilidad" para filtrar opciones. 4. El sistema presenta al usuario los resultados de la búsqueda.
Flujos Alternativos	• **Excepción 1:** No existen laboratorios disponibles para el criterio o fecha seleccionada.
Reglas de Negocio	Los horarios mostrados deben excluir franjas previamente reservadas o deshabilitadas.
Datos Involucrados	• Nombre_Laboratorio. • Capacidad. • Equipamiento.

	• Fecha. • Horarios_Libres.
GUI Asociada	Vista/Buscador de Disponibilidad de Laboratorios.

Tabla 9. *Especificación del Caso de Uso CU-02 (Validar disponibilidad).*

Especificación	**Detalle**
ID	CU-02
Nombre	Validar disponibilidad
Actor(es)	BD_Laboratorios
Objetivo	Comprobar de manera lógica y en base de datos si una franja horaria de un laboratorio específico está libre de traslapes.
Precondiciones	Datos de consulta (ID_Laboratorio, Fecha, Horario) provistos.
Postcondiciones	El sistema determina si la solicitud es válida (disponible) o rechazada (no disponible).
Flujo Principal	1. El sistema recibe la solicitud de verificación con fecha y hora. 2. El sistema consulta en BD_Laboratorios las reservas coincidentes. 3. El sistema valida que no existan cruces de horarios. 4. El sistema retorna el estado de disponibilidad como positivo.
Flujos Alternativos	• **Excepción 1:** Se detecta un solapamiento con una reserva existente. El sistema retorna resultado negativo.
Reglas de Negocio	Un laboratorio no puede ser asignado a más de un usuario dentro de un mismo bloque de tiempo.
Datos Involucrados	• D_Laboratorio. • Fecha_Solicitada.

	• Hora_Solicitada. • Estado_Validacion.
GUI Asociada	N/A (Proceso interno/Lógica del sistema).

Tabla 10. *Especificación del Caso de Uso CU-02 (Confirmar reserva).*

Especificación	**Detalle**
ID	CU-02
Nombre	Confirmar reserva
Actor(es)	Usuario, BD_Reservas
Objetivo	Formalizar la reserva del laboratorio por parte del usuario y registrarla de forma definitiva.
Precondiciones	La disponibilidad del laboratorio debió ser validada previamente como disponible.
Postcondiciones	La reserva queda guardada en estado "Confirmada" en BD_Reservas.
Flujo Principal	1. El usuario presiona el botón para confirmar la reserva. 2. El sistema genera los detalles finales del comprobante de reserva. 3. El sistema almacena el registro en BD_Reservas. 4. El sistema emite un mensaje de éxito y número de comprobante al usuario.
Flujos Alternativos	• **Excepción 1:** El usuario cancela el proceso de confirmación antes de guardar. • **Excepción 2:** Error al guardar en BD_Reservas.
Reglas de Negocio	Una reserva confirmada debe generar un ID único de reserva.
Datos Involucrados	• ID_Reserva.

	• ID_Usuario. • ID_Laboratorio. • Fecha. • Horario. • Fecha_Creacion.
GUI Asociada	Pantalla de Confirmación de Reserva / Comprobante.

Tabla 11. *Especificación del Caso de Uso CU-02 (Actualizar reserva).*

Especificación	**Detalle**
ID	CU-02
Nombre	Actualizar reserva
Actor(es)	Usuario, BD_Reservas
Objetivo	Modificar los parámetros (fecha, horario, laboratorio) de una reserva previamente efectuada.
Precondiciones	Debe existir una reserva activa asociada al usuario en BD_Reservas.
Postcondiciones	La reserva queda actualizada con los nuevos datos en BD_Reservas.
Flujo Principal	1. El usuario selecciona una reserva activa de su lista. 2. El usuario modifica los campos deseados (p. ej., cambio de fecha u hora). 3. El sistema valida la disponibilidad de la nueva fecha/hora seleccionada. 4. El sistema actualiza los registros en BD_Reservas. 5. . El sistema notifica la modificación exitosa.
Flujos Alternativos	• **Excepción 1:** El nuevo horario no está disponible. No se aplican los cambios.

Reglas de Negocio	Las modificaciones deben realizarse con un tiempo mínimo de antelación respecto a la cita original.
Datos Involucrados	• D_Reserva. • Datos_Nuevos (Fecha, Hora, Laboratorio).
GUI Asociada	Formulario de Edición de Reserva.

Tabla 12. *Especificación del Caso de Uso CU-02 (Cancelar reserva).*

Especificación	**Detalle**
ID	CU-02
Nombre	Cancelar reserva
Actor(es)	Usuario, BD_Reservas
Objetivo	Anular una reserva previamente programada por el usuario.
Precondiciones	Existencia de una reserva en estado activa en BD_Reservas.
Postcondiciones	La reserva pasa a estado "Cancelada" y el horario de dicho laboratorio queda liberado.
Flujo Principal	1. El usuario selecciona la reserva que desea cancelar. 2. El sistema solicita confirmación de la cancelación. 3. El usuario confirma la cancelación. 4. El sistema modifica el estado de la reserva a "Cancelada" en BD_Reservas. 5. El sistema muestra la notificación de anulación correcta.
Flujos Alternativos	• **Excepción 1:** El usuario desiste de cancelar y la reserva se mantiene activa.

Reglas de Negocio	Las cancelaciones fuera de plazo estipulado pueden estar sujetas a restricciones o sanciones.
Datos Involucrados	• ID_Reserva, Motivo_Cancelacion, • Estado_Reserva ("Cancelada").
GUI Asociada	Vista de Mis Reservas / Ventana de Confirmación de Cancelación.

Tabla 13. *Especificación del Caso de Uso CU-02 (Crear Historial).*

Especificación	**Detalle**
ID	CU-02
Nombre	Crear historial
Actor(es)	BD_Reservas
Objetivo	Registrar la traza, auditoría y eventos sucedidos con las reservas (creación, edición, cancelación, finalización).
Precondiciones	Ocurrencia de cualquier acción sobre una reserva dentro del sistema.
Postcondiciones	La entrada de auditoría queda persistida en la tabla de historial de BD_Reservas.
Flujo Principal	1. El sistema detecta una acción realizada sobre una reserva (ej. creación, edición o cancelación). 2. El sistema recopila los datos del evento (ID_Reserva, Tipo de Acción, Timestamp, Usuario). 3. El sistema guarda la entrada en la base de datos BD_Reservas.
Flujos Alternativos	• **Excepción 1:** Error de conexión o escritura en la base de datos.
Reglas de Negocio	Los registros del historial son de solo lectura y no pueden ser borrados ni modificados por el usuario.

Datos Involucrados	• ID_Historial. • ID_Reserva. • Tipo_Evento. • Fecha_Hora_Registro. • ID_Usuario.
GUI Asociada	N/A (Proceso interno/Lógica de auditoría).

Tabla 14. *Especificación del Caso de Uso CU-02 (Consultar Historial).*

Especificación	**Detalle**
ID	CU-02
Nombre	Consultar historial
Actor(es)	Usuario, BD_Reservas
Objetivo	Permitir al usuario visualizar el registro de reservas pasadas, actuales y sus estados históricos.
Precondiciones	El usuario debe estar autenticado en la plataforma.
Postcondiciones	El sistema despliega el reporte histórico de actividades asociadas al usuario.
Flujo Principal	El usuario ingresa a la sección de historial de reservas. El sistema organiza y renderiza los datos en pantalla ordenados cronológicamente.
Flujos Alternativos	Excepción 1: El usuario no posee registros históricos asociados. El sistema muestra un mensaje indicando que no hay datos.
Reglas de Negocio	Cada usuario solo puede visualizar su propio historial de reservas salvo que posea un rol administrativo.

Datos Involucrados	Listado de ID_Reserva, Fechas, Estado (Completada, Cancelada, etc.), Laboratorio.
GUI Asociada	Pantalla / Tablero de Historial de Reservas.

Figura 3.1. *Diagrama de Secuencia (Gestión Reserva)*

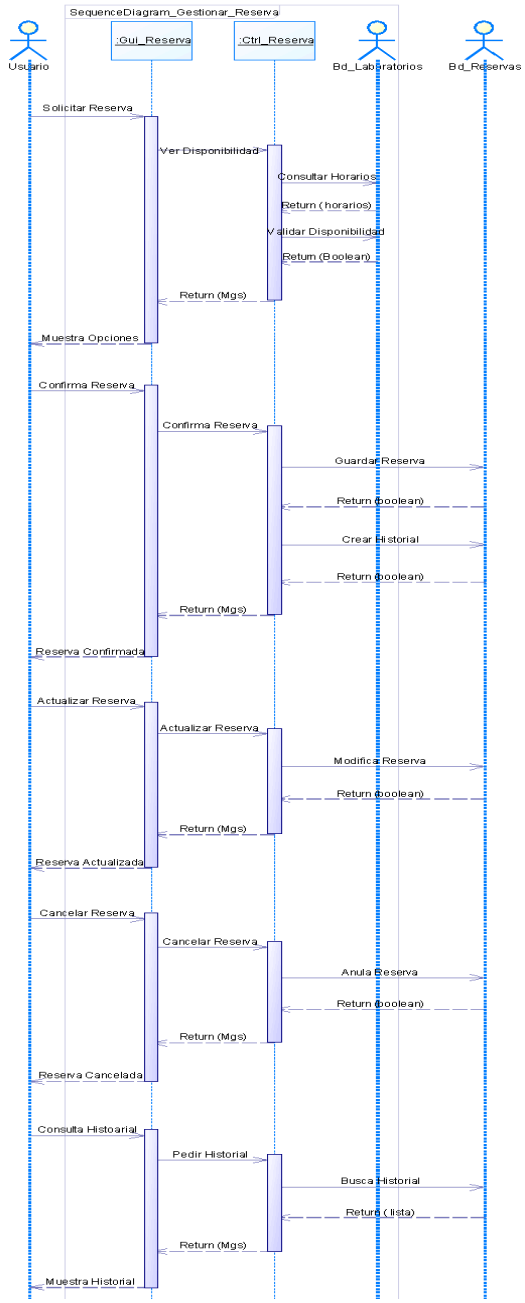


Tabla 14.1 *Especificación del Diagrama Secuencia (Gestionar Reserva).*

Especificación	**Detalle**
Caso de uso	Gestionar reserva
ID	CU-02
Actores	Usuario, Bd_Laboratorios, Bd_Reservas
Objetos	:Gui_Reserva, :Ctrl_Reserva
Precondición	El usuario ha iniciado sesión.
Postcondición	La reserva queda confirmada, actualizada o cancelada en BD_Reservas, y el historial se crea o se consulta.
Descripción del flujo	El usuario solicita una reserva y el sistema consulta la disponibilidad en Bd_Laboratorios, validando antes de mostrar las opciones. Luego el usuario confirma la reserva, que se guarda en Bd_Reservas junto con la creación del historial. Después, el usuario puede elegir una de tres acciones, representadas en el fragmento alt, actualizar la reserva (mensajes 17 al 22), cancelarla (23 al 28) o consultar su historial (29 al 34). Las tres operan sobre Bd_Reservas.

Figura 4. *Caso de Uso Gestionar Reserva desde Administración.*

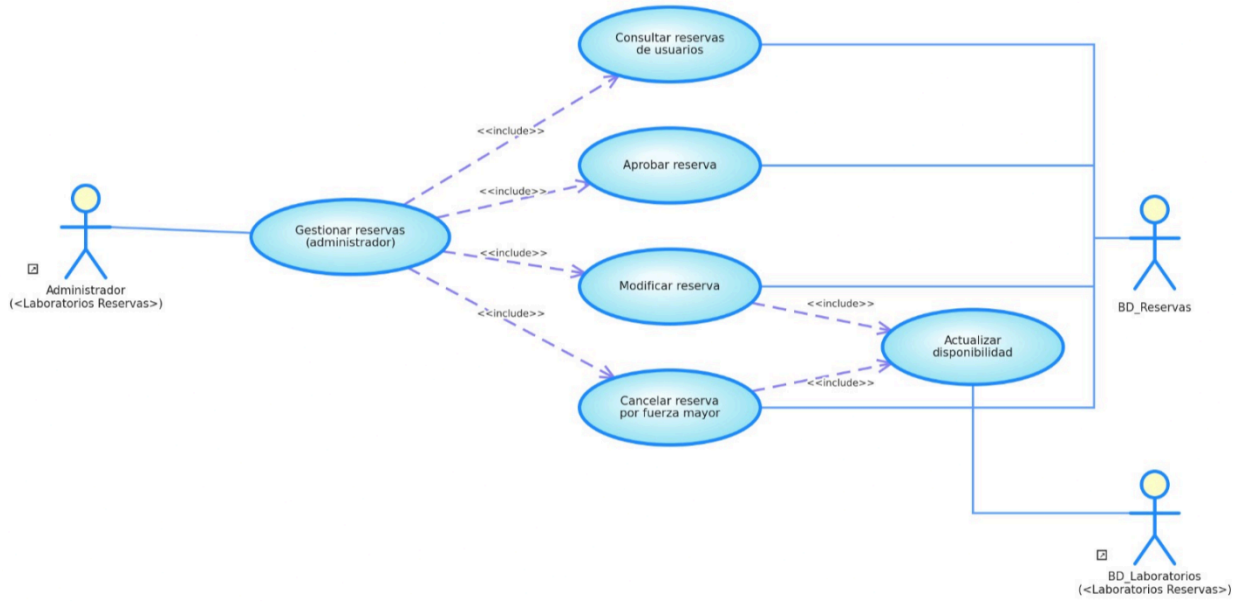


Tabla 15. *Especificación del Caso de Uso CU-03 (Gestionar Reserva Admin).*

Especificación	Detalle
ID	CU-03
Nombre	Gestionar Reservas (Administrador)
Actor(es)	Administrador, BD_Reservas, BD_Laboratorios
Objetivo	Permitir al administrador supervisar las solicitudes de reservas de los usuarios, pudiendo aprobarlas, modificarlas o cancelarlas por fuerza mayor, asegurando la actualización de la disponibilidad del sistema en las bases de datos.
Precondiciones	El administrador debe haberse autenticado en el sistema con credenciales de rol administrativo.
Postcondiciones	Las reservas gestionadas cambian su estado (aprobada, modificada o cancelada) en BD_Reservas y se actualiza la disponibilidad de horarios/laboratorios en BD_Laboratorios cuando corresponde.
Flujo Principal	1. El administrador accede al módulo de administración de reservas. 2. El sistema consulta a BD_Reservas y despliega la lista de reservas registradas por los usuarios (incluye Consultar reservas de usuarios). 3. El administrador selecciona una solicitud de reserva pendiente de aprobación. 4. El administrador procesa y aprueba la solicitud (incluye Aprobar reserva). 5. El sistema actualiza el estado de la reserva en BD_Reservas y notifica al usuario la confirmación.
Flujos Alternativos	**Modificar reserva:** ● En el paso 3, el administrador selecciona una reserva para modificar sus parámetros (ej. cambio de horario o laboratorio asignado). ● El administrador aplica los cambios requeridos (incluye Modificar reserva). ● El sistema invoca el proceso Actualizar disponibilidad para validar y reflejar los cambios en BD_Laboratorios y BD_Reservas. ● El sistema notifica la actualización. **Cancelar reserva por fuerza mayor:**

	• En el paso 3, el administrador decide anular una reserva debido a un evento de fuerza mayor (ej. mantenimiento imprevisto del laboratorio). • El administrador ingresa el motivo y confirma la anulación (incluye Cancelar reserva por fuerza mayor). • El sistema invoca el proceso Actualizar disponibilidad para liberar el horario del laboratorio en BD_Laboratorios y actualizar el estado en BD_Reservas. • El sistema emite la notificación correspondiente a los usuarios afectados. **Conflicto al actualizar disponibilidad:** Si al modificar una reserva se detecta un solapamiento con otro evento en BD_Laboratorios, el sistema rechaza la modificación y advierte al administrador. **Fallo en base de datos:** Si existe un error de conexión con BD_Reservas o BD_Laboratorios, la transacción se revierte y se muestra un mensaje de error
Reglas de Negocio	• Solo los usuarios con rol de Administrador pueden autorizar, modificar o cancelar reservas de terceros por causa mayor. • Toda modificación o cancelación por fuerza mayor debe actualizar obligatoriamente el catálogo de disponibilidad en BD_Laboratorios. • La aprobación o cancelación efectuada debe registrar el motivo y la firma/ID del administrador responsable.
Datos Involucrados	• ID_Reserva • ID_Usuario • ID_Administrador • ID_Laboratorio • Fecha_Reserva • Horario_Inicio • Horario_Fin • Estado_Reserva (Pendiente, Aprobada, Modificada, Cancelada por Fuerza Mayor). • Motivo_Cancelacion.
GUI Asociada	Panel de Control Administrativo de Reservas (incluye Lista de Solicitudes de Usuarios, Formulario de Edición/Aprobación y Ventana Modal de Cancelación por Fuerza Mayor).

Figura 4.1. *Diagrama Secuencia Gestión Reservas desde Administración.*

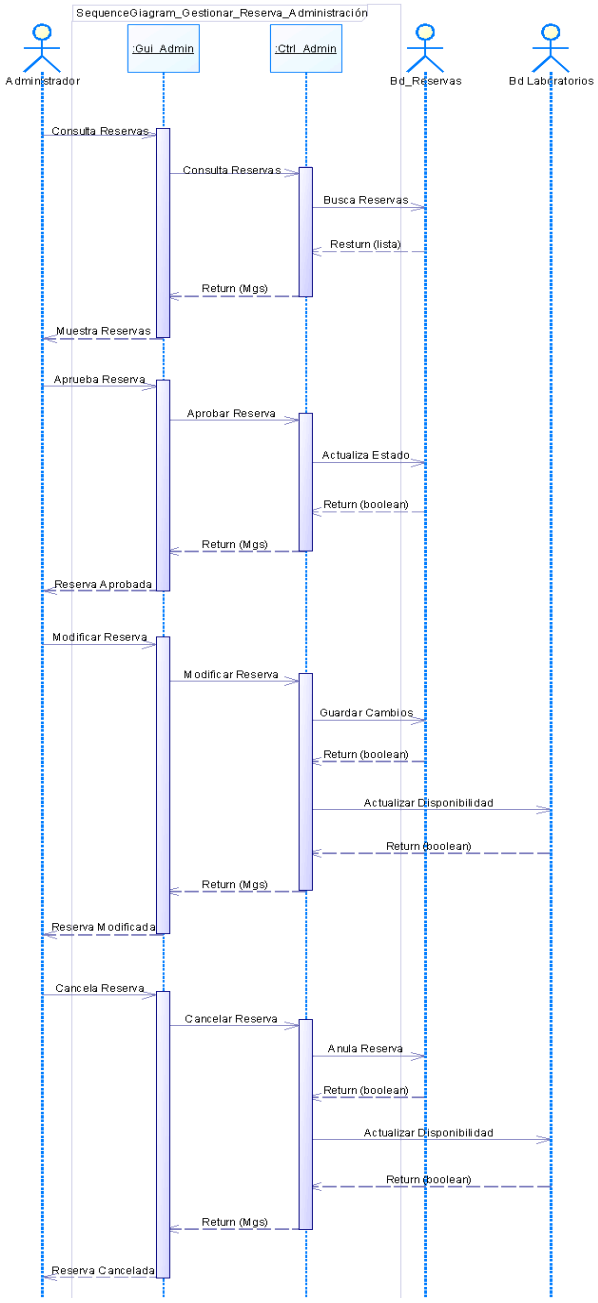


Tabla 15.1. *Especificación del Diagrama de Secuencia (Gestión Reservas desde Admin).*

Especificación	Detalle
Caso de uso	Gestionar reserva desde Administración
ID	CU_03
Actores	Administrador, Bd_Reservas, Bd_Laboratorios
Objetos	:Gui_Admin, :Ctrl_Admin
Precondición	El administrador ha iniciado sesión.
Postcondición	La reserva queda aprobada, modificada o cancelada; si se modifica o se cancela, la disponibilidad se actualiza en BD_Laboratorios.
Descripción del flujo	El administrador consulta las reservas de los usuarios en Bd_Reservas. A partir de esa consulta puede aprobar, modificar o cancelar una reserva, representado en el fragmento alt. Aprobar actualizar el estado de la reserva. Modificar guarda los cambios y actualiza la disponibilidad en Bd_Laboratorios. Cancelar por fuerza mayor anula la reserva y también actualiza la disponibilidad.

Figura 5. *Caso de Uso Generar Estadísticas y Reportes.*

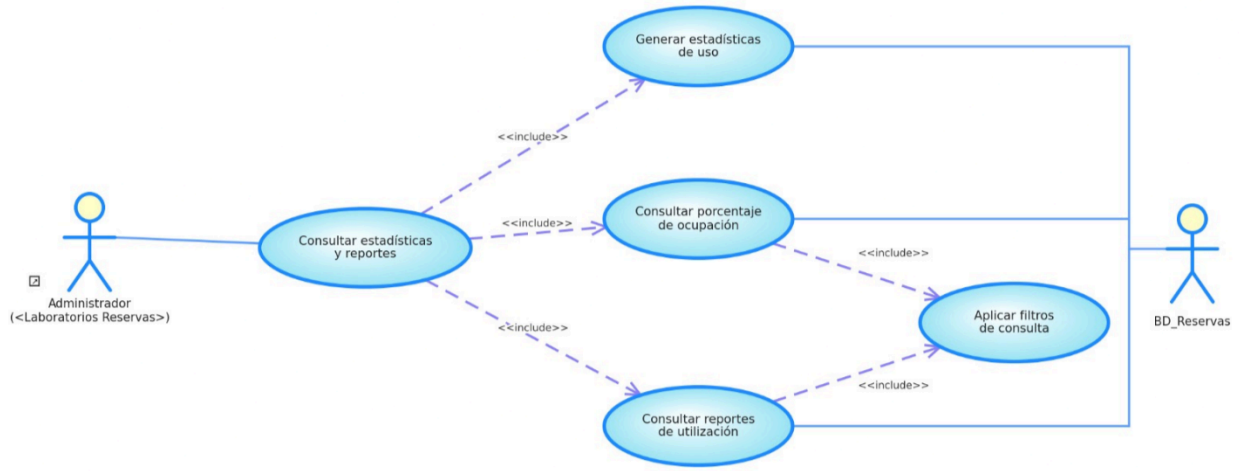


Tabla 16. *Especificación del Caso de Uso CU-04 (Consultar estadísticas y reportes).*

Especificación	Detalle
ID	CU-04
Nombre	Consultar estadísticas y reportes
Actor(es)	Administrador, BD_Reservas
Objetivo	Generar estadísticas, porcentajes de ocupación y reportes de uso de los laboratorios.
Precondiciones	El administrador debe estar autenticado y con permisos en el sistema.
Postcondiciones	Se muestran en pantalla las estadísticas o reportes generados.
Flujo Principal	1. El administrador ingresa a la opción de estadísticas y reportes. 2. Selecciona el tipo de informe (Estadísticas de uso, Porcentaje de ocupación o Reporte de utilización). 3. Aplica los filtros de consulta deseados. 4. El sistema consulta los datos en la base de datos (BD_Reservas). 5. El sistema procesa la información y muestra los resultados.
Flujos Alternativos	• **Excepción 1:** No existen datos para los filtros seleccionados. • **Excepción 2:** Error de conexión con la base de datos. • **Excepción 3:** Filtros de fecha inválidos (ej. fecha de inicio mayor a la fecha de fin).
Reglas de Negocio	Acceso exclusivo para el rol de Administrador.
Datos Involucrados	• Filtros (fechas, laboratorio). • Métricas de uso. • Porcentaje de ocupación.

GUI Asociada	Panel administrativo (futura).

Figura 5.1. *Diagrama Secuencia Generar Estadísticas y Reportes.*

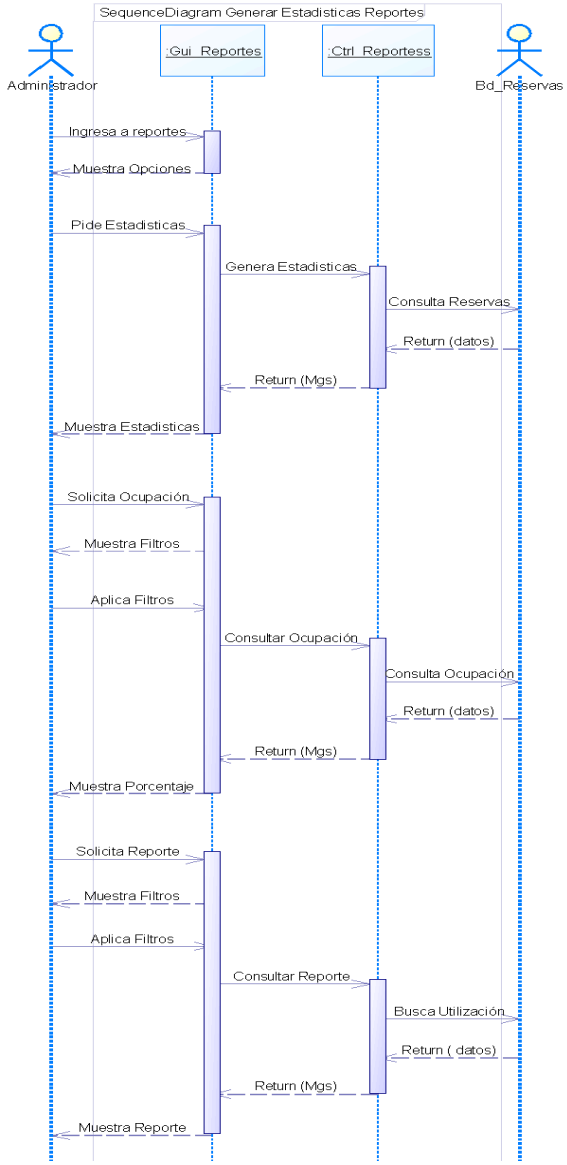


Tabla 16.1. *Especificación del Diagrama de Secuencia (Generar estadísticas y reportes).*

Especificación	Detalle
Caso de uso	Generar Estadísticas y Reportes
ID	CU-04
Actores	Administrador, Bd_Reservas
Objetos	:Gui_Reportes, :Ctrl_Reportes
Precondición	El administrador ha iniciado sesión.
Postcondición	Se muestran las estadísticas o los reportes consultados.
Descripción del flujo	El administrador ingresa a la opción de reportes y el sistema le muestra la alternativa. Según su elección, el fragmento alt cubre tres ramas, todas con consulta a Bd_Reservas. Estadísticas de uso (mensajes 3 al 8), porcentaje de ocupación (9 al 16) y reportes de utilización (17 al 24). Las dos últimas incluyen el paso, Aplicar filtros de consulta (mensajes 10 y 11, y 18 y 19).

Figura 6. *Caso de Uso Consumir API REST.*

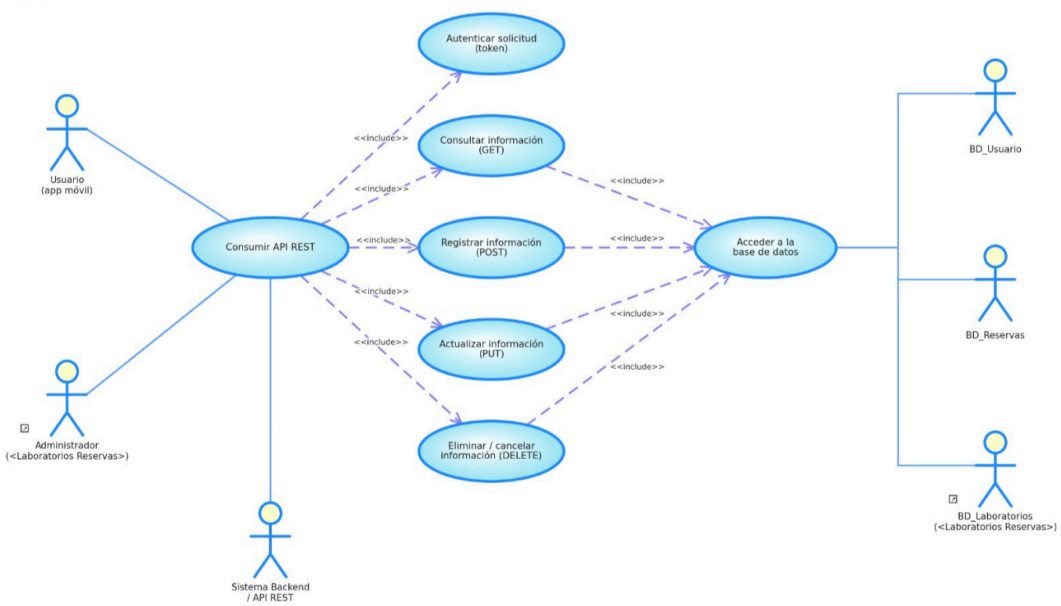
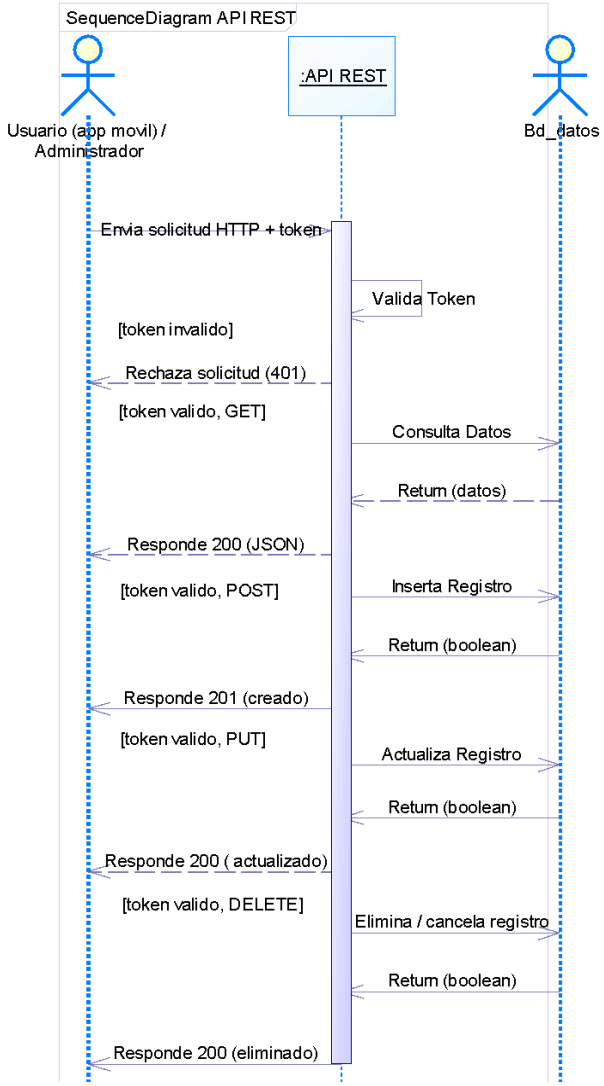


Tabla 17. *Especificación del Caso de Uso CU-05 (Consumir API REST).*

Especificación	Detalle
ID	CU-05
Nombre	Consumir API REST
Actor(es)	Principales*:* Usuario (app móvil), Administrador, Sistema Backend / API REST Secundarios*:* BD_Usuario, BD_Reservas, BD_Laboratorios
Objetivo	Ejecutar peticiones (GET, POST, PUT, DELETE) para gestionar la información del sistema mediante la API REST.
Precondiciones	La petición debe contener un token de autenticación válido.
Postcondiciones	La base de datos correspondiente procesa la acción
Flujo Principal	1. El cliente (Usuario, Administrador o Sistema Backend) envía una solicitud HTTP a la API REST. 2. La API auténtica la solicitud mediante el token. 3. Según la acción enviada (GET, POST, PUT o DELETE), la API procesa la petición: a. **GET:** Consulta de información. b. **POST:** Registro de nueva información. c. **PUT:** Actualización de información existente. d. **DELETE:** Eliminación o cancelación de información. e. La API accede a la base de datos correspondiente (BD_Usuario, BD_Reservas o BD_Laboratorios). 4. La base de datos ejecuta la operación y la API retorna la respuesta al cliente.
Flujos Alternativos	• Token de autenticación inválido o expirado. • Parámetros o formato de la petición incorrectos.

	• Error de conexión o fallo de consulta con las bases de datos (BD_Usuario, BD_Reservas, BD_Laboratorios). • Recurso solicitado no encontrado.
Reglas de Negocio	• Todas las peticiones deben validar el token de autorización antes de operar. • Solo los roles autorizados pueden ejecutar métodos modificadores (POST, PUT, DELETE) sobre recursos protegidos.
Datos Involucrados	Token de autenticación, parámetros de consulta, cuerpo de la petición (formato REST) y datos de bases de datos (BD_Usuario, BD_Reservas, BD_Laboratorios).
GUI Asociada	No aplica

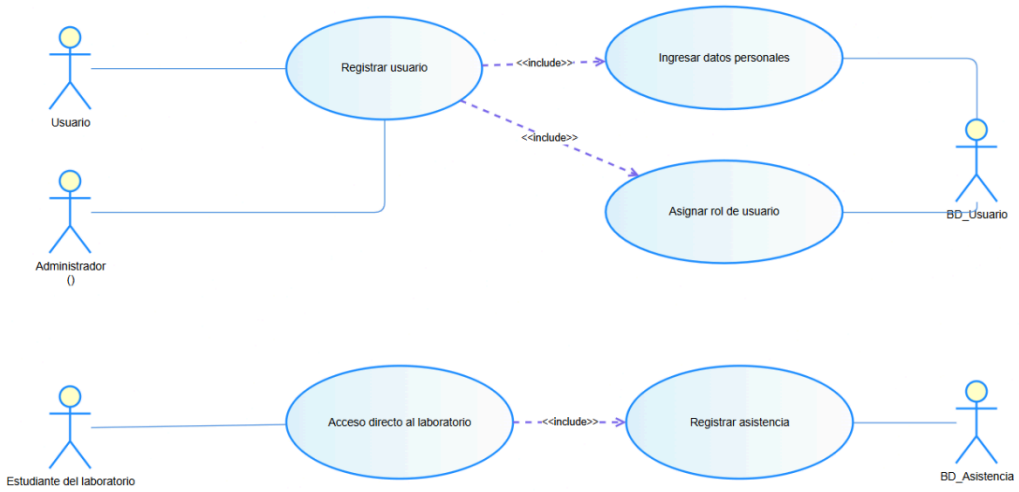
Figura 6.1. *Diagrama de Secuencia Consumir API REST.*



Tablas 18. *Especificación del Diagrama de Secuencia (Consumir API REST).*

Especificación	**Detalle**
Caso de uso	Consumir API REST
ID	CU-05
Actores	Usuario (app móvil) / Administrador, Base de Datos
Objetos	API_REST
Precondición	El cliente (app móvil o administrador) cuenta con un token de acceso.
Postcondición	La API responde con el resultado de la operación solicitada.
Descripción del flujo	El cliente envía una solicitud HTTP con su token a la API REST, que valida el token (mensajes 1 y 2). Si el token no es válido, la API rechaza la solicitud con un error 401 (mensaje 3). Si es válido, según el método ejecuta una operación sobre la base de datos: consulta (GET, mensajes 4 al 6), inserción (POST, 7 al 9), actualización (PUT, 10 al 12) o eliminación/cancelación (DELETE, 13 al 15). Cada par de mensajes hacia Base_Datos corresponde a Acceder a la base de datos.

Figura 7. *Caso de Uso Gestión y asignación de rol de usuario.*



Tablas 18. *Especificación del Caso de Uso CU-06 (C).*

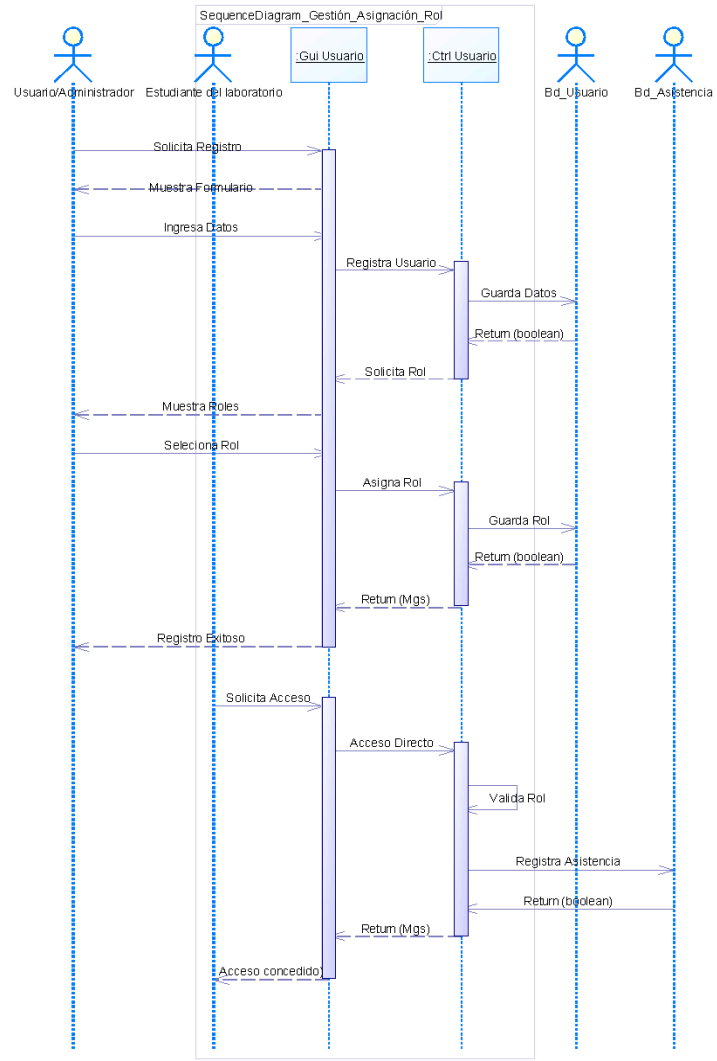
Especificación	Detalle
ID	CU-06
Nombre	Asignación de rol de usuario
Actor(es)	**Principales*:*** Usuario, Administrador **Secundario:** BD_Usuario
Objetivo	Dar de alta un nuevo usuario en el sistema ingresando sus datos personales y asignándole un rol.
Precondiciones	El administrador o usuario debe tener acceso al módulo de registro en el sistema.
Postcondiciones	El usuario queda registrado en BD_Usuario con sus datos personales y rol correspondientes.
Flujo Principal	1. El actor (usuario o administrador) inicia el proceso de registro. 2. El sistema solicita el ingreso de los datos personales (ingresar datos personales). 3. El actor ingresa los datos personales requeridos. 4. El sistema asigna el rol correspondiente al usuario (asignar rol de usuario). 5. El sistema guarda la información en la base de datos BD_Usuario y confirma el registro exitoso.
Flujos Alternativos	● Datos personales incompletos o con formato inválido. ● El usuario ya se encuentra registrado en el sistema. ● Error de conexión con BD_Usuario.

Reglas de Negocio	• Todo usuario registrado debe poseer obligatoriamente un rol asignado para definir sus permisos.
DatosInvolucrados	• Datos personales (nombre, documento, correo, etc.), rol de usuario.
GUI Asociada	Panel Administrativo (futura)

Especificación	**Detalle**
ID	CU-06
Nombre	Acceso directo al laboratorio
Actor(es)	**Principales*:*** Usuario, Administrador **Secundario:** BD_Asistencia
Objetivo	Permitir el ingreso directo del estudiante al laboratorio registrando su asistencia en el sistema.
Precondiciones	El estudiante debe estar previamente matriculado o autorizado para ingresar al laboratorio.
Postcondiciones	Se guarda el registro de asistencia del estudiante en la base de datos BD_Asistencia.
Flujo Principal	1. El estudiante del laboratorio solicita o realiza el acceso directo al laboratorio. 2. El sistema ejecuta el registro de asistencia (Registrar asistencia). 3. El sistema valida los datos del estudiante y almacena la fecha/hora de ingreso en BD_Asistencia. 4. El sistema confirma la asistencia y permite el acceso al laboratorio.

Flujos Alternativos	• El estudiante no está habilitado para acceder al laboratorio. • Error al registrar la asistencia en BD_Asistencia.
Reglas de Negocio	No se puede dar acceso al laboratorio sin haber registrado previamente la asistencia del estudiante.
Datos Involucrados	Identificación del estudiante, fecha, hora y laboratorio de acceso.
GUI Asociada	Panel Administrativo (futura)

Figura 7.1. *Diagrama de Secuencia Gestión y asignación de rol de usuario.*



Tablas 18.1. *Especificación Diagrama de Secuencia Gestión y Asignación de rol*

Especificación	Detalle
Caso de uso	Figura 7. Caso de uso Gestión y asignación de rol de usuario
Historia de usuario	HU-F31 (Gestión y asignación de rol de usuario); CU_06
Actores	Usuario / Administrador, Estudiante del laboratorio, Bd_Usuario, Bd_Asistencia
Objetos	:Gui_Usuario, :Ctrl_Usuario
Precondición	Para el registro, el usuario aún no está registrado; para el acceso directo, el estudiante cuenta con el rol Estudiante del laboratorio.
Postcondición	Los datos personales y el rol quedan almacenados en BD_Usuario, la asistencia del estudiante queda almacenada en BD_Asistencia.
Descripción del flujo	La figura 7 reúne dos casos de uso. En el primero, el usuario o el administrador registra un usuario: ingresa sus datos personales, que se guardan en Bd_Usuario, y asigna el rol Egresado o Estudiante del laboratorio, que también se guarda en Bd_Usuario. En el segundo, el estudiante del laboratorio accede de forma directa: el sistema valida su rol y registra la asistencia en Bd_Asistencia.

.

Figura 8. *Caso de Uso Gestión de Acceso, Asistencia y Control de Horas de Laboratorio.*

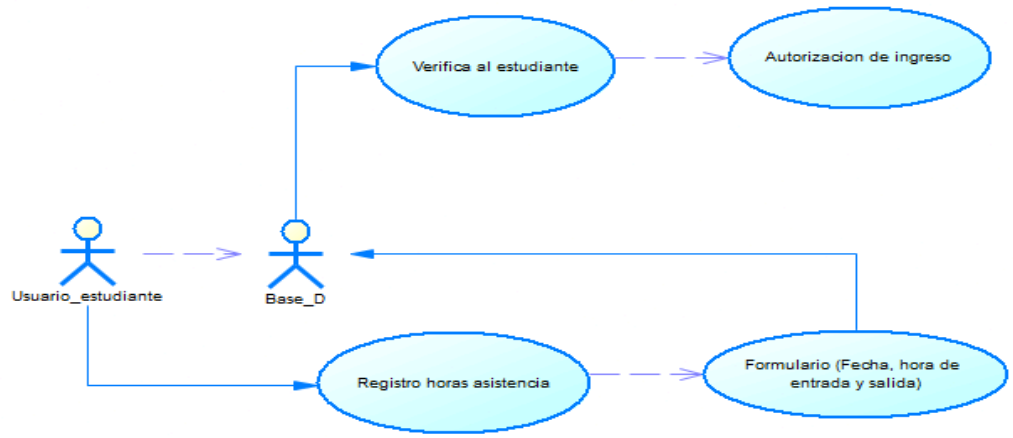


Tabla 19. *Especificación del Caso de Uso CU-07 (Gestión de Acceso, Asistencia y Control de Horas de Laboratorio).*

Especificación	Detalle
ID	CU-07
Nombre	Gestión de Acceso, Asistencia y Control de Horas de Laboratorio
Actor(es)	Usuario Estudiante, Base de datos (BD)
Objetivo	Permitir que el estudiante acceda directamente a los espacios y recursos que brinda el laboratorio sin reserva previa, y registre sus horas de asistencia (fecha, hora de entrada y hora de salida) para el seguimiento y control de su permanencia.
Precondiciones	El usuario deberá estar registrado y verificado como estudiante en la BD El estudiante contará con autorización permanente como miembro del laboratorio
Postcondiciones	Autorizar el ingreso del estudiante sin necesidad de reserva Almacenar la asistencia del usuario estudiante
Flujo Principal	1. El estudiante ingresa al sistema y solicita acceso al laboratorio. 2. El sistema consulta la BD y verifica que sea un estudiante del laboratorio (include). 3. El sistema emite la autorización de ingreso sin requerir reserva previa. 4. El estudiante selecciona la opción de registro de horas de asistencia (include). 5. El sistema muestra el formulario de registro (fecha, hora de entrada y hora de salida). 6. El estudiante diligencia el formulario, o el sistema toma la marcación en tiempo real. 7. El sistema valida los datos y los almacena en la BD. 8. El sistema confirma el registro exitoso de la asistencia.
Flujos Alternativos	• **Estudiante no verificado:** Si el usuario no existe en la BD o no tiene el rol de Estudiante, el sistema deniega el ingreso y muestra un mensaje.

	• **Datos incompletos o inválidos:** Si faltan campos o la hora de salida es anterior a la de entrada, el sistema informa el error y solicita corregir el formulario. • **Marcación en tiempo real:** El estudiante marca entrada y salida en el momento, y el sistema registra automáticamente la fecha y la hora. • **Falla de conexión con la BD:** el sistema notifica el error y no guarda el registro; el estudiante debe intentarlo de nuevo.
Reglas de Negocio	• El estudiante del laboratorio tiene autorización permanente, por lo que no necesita una reserva previa para ingresar (HU-F32). • Solo los usuarios con rol «Estudiante del laboratorio» pueden usar este caso de uso. • Todo registro de asistencia debe incluir fecha, hora de entrada y hora de salida (HU-F33). • La hora de salida debe ser posterior a la hora de entrada. • La información de asistencia se almacena en la BD para el seguimiento de horas y el control de permanencia
Datos Involucrados	**Entrada:** Identificación del estudiante, fecha, hora de entrada, hora de salida. **Consulta:** Rol y estado de autorización del estudiante. **Salida:** Resultado de la verificación (autorizado o denegado), confirmación del registro, historial y total de horas acumuladas.
GUI Asociada	Vista Inicio (futura), Panel Estudiante (futura), Vista Registro de Asistencia (futura), Mis Reservas e Historial (futura).

Figura 8.1. *Diagrama de Secuencia Gestión de Acceso, Asistencia y Control de Horas de Laboratorio*

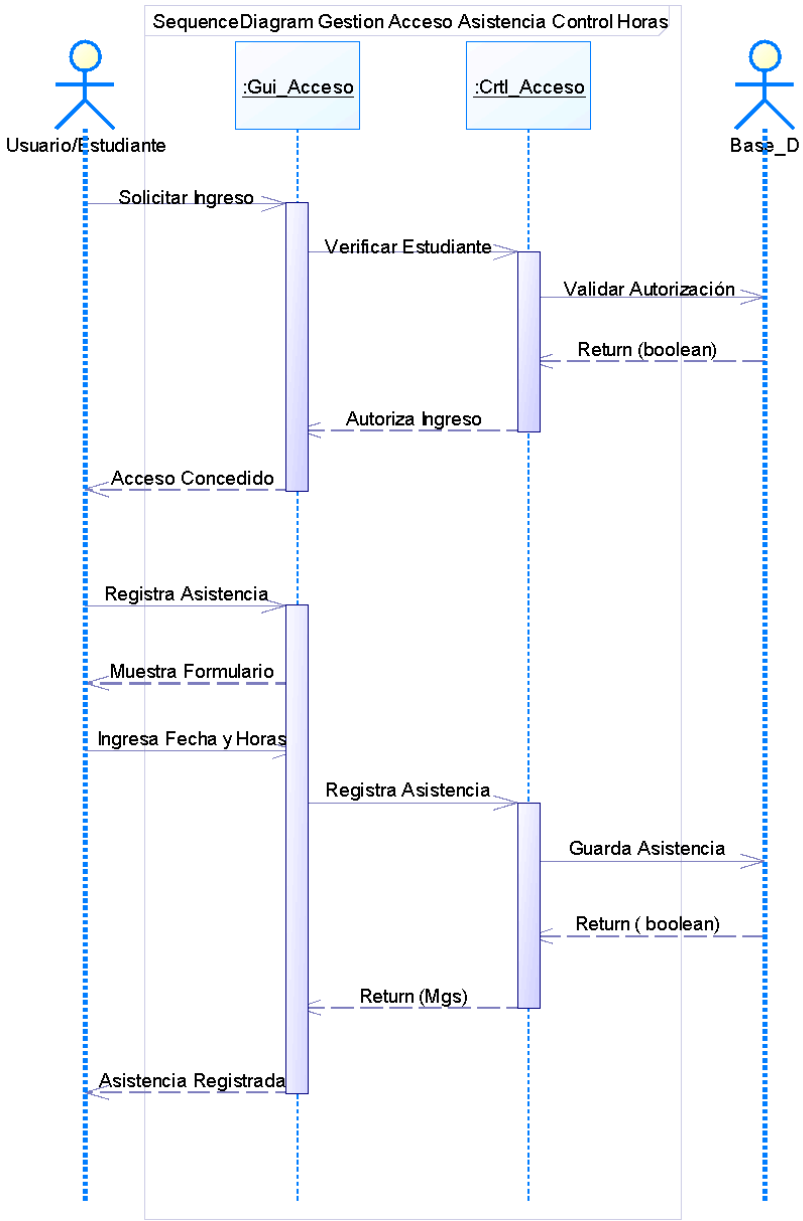


Tabla 19. *Especificación de Diagrama De Secuencia (Gestión de Acceso, Asistencia y Control de Horas de Laboratorio).*

Especificación	**Detalle**
Caso de uso	Figura 8. Caso de uso Gestión de Acceso, Asistencia y Control de Horas de Laboratorio
ID	CU_07
Actores	Usuario/estudiante (Estudiante del laboratorio), Base_D
Objetos	:Gui_Acceso, :Ctrl_Acceso
Precondición	El usuario está registrado con el rol Estudiante del laboratorio y tiene autorización permanente.
Postcondición	Se autoriza el ingreso y la asistencia (fecha, hora de entrada y de salida) queda almacenada en la base de datos.
Descripción del flujo	El estudiante solicita el ingreso al laboratorio. La interfaz (:Gui_Acceso) envía la solicitud al controlador (:Ctrl_Acceso), que consulta en Base_D si el estudiante tiene autorización. Si está autorizado, el sistema concede el acceso sin consultar disponibilidad ni exigir una reserva previa. Luego el estudiante registra su asistencia: el sistema muestra el formulario, el estudiante ingresa la fecha y las horas de entrada y salida, y el controlador las guarda en Base_D. Al confirmarse el guardado, el sistema informa que la asistencia fue registrada.

7.2 Control de versiones (Git Hub)

Rama	Propósito
main	Código estable; recibe los PR de las ramas de trabajo
develop	Integración; hoy solo tiene el README
backend	Desarrollo de la API
frontend	Desarrollo de la app Flutter
feature/login-registro	Registro, login y recuperación de contraseña (PR #1, 6 de octubre de 2026)
feature/reserva	Laboratorios y reservas (PR #2, 9 de octubre de 2026)

Rama	Propósito
feature/aplicativofuncional	Aplicativo integrado con Docker
pruebas	Backend y colecciones Postman de QA
Documentos	Casos de uso y diagramas de secuencia

8. Desarrollo

Código fuente relevante

Backend

Conexión a PostgreSQL

```
const { Pool } = require("pg");
const dotenv = require("dotenv");

dotenv.config();

const pool = new Pool({
   host: process.env.DB_HOST,
   port: process.env.DB_PORT,
   database: process.env.DB_NAME,
   user: process.env.DB_USER,
   password: process.env.DB_PASSWORD
});
module.exports = pool;
```

Aplicación Express

```
const app = express();
app.use(express.json());

app.use("/api/auth", authRoutes);
```

```
app.use("/api/reservas", reservaRoutes);
app.use("/api/asistencia", asistenciaRoutes);
app.use("/api/horarios", horarioRoutes);
app.use("/api/laboratorios", laboratorioRoutes);

module.exports = app;
```

Validación del token JWT

```
const verificarToken = (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader) {
      return res.status(401).json({ mensaje: "Token no proporcionado" });
    }

    const [tipo, token] = authHeader.split(" ");
    if (tipo !== "Bearer" || !token) {
      return res.status(401).json({ mensaje: "Formato de token inválido" });
    }

    req.usuario = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (error) {
    return res.status(401).json({ mensaje: "Token inválido o expirado" });
  }
};
```

Autorización por rol

```
const autorizarRoles = (...rolesPermitidos) => {
   return (req, res, next) => {
      if (!req.usuario) {
         return res.status(401).json({ mensaje: "Usuario no autenticado" });
      }
      const rolUsuario = (req.usuario.rol || "").toUpperCase();
      const tipoUsuario = (req.usuario.tipo_usuario || "").toUpperCase();
      const rolesNormalizados = rolesPermitidos.map(r => r.toUpperCase());

      const tienePermiso = rolesNormalizados.includes(rolUsuario) ||
                           rolesNormalizados.includes(tipoUsuario);
      if (!tienePermiso) {
         return res.status(403).json({ mensaje: `Acceso denegado: ...` });
      }
      next();
   };
};
```

Solo estudiantes

```
const resultado = await pool.query(
   `SELECT tipo_usuario FROM usuario WHERE id_usuario = $1`,
   [idUsuario]
);
// ... si no existe el usuario: 401
if (resultado.rows[0].tipo_usuario !== "ESTUDIANTE") {
   return res.status(403).json({
      mensaje: "El control de asistencia está disponible únicamente para estudiantes."
```

```
    });
}
next();
```

Registro de usuarios

```
if (!/^\d{10}$/.test(telefono)) {
    return res.status(400).json({
        mensaje: "El teléfono debe tener exactamente 10 dígitos"
    });
}
if (contrasena !== confirmarContrasena) {
    return res.status(400).json({ mensaje: "Las contraseñas no coinciden" });
}
if (contrasena.length < 8) {
    return res.status(400).json({
        mensaje: "La contraseña debe tener al menos 8 caracteres"
    });
}
if (!["EGRESADO", "ESTUDIANTE"].includes(tipo_usuario)) {
    return res.status(400).json({
        mensaje: "Debes seleccionar un tipo de persona válido"
    });
}
// ... se consulta si el correo o el teléfono ya existen (409)
```

Inicio de sesión

```
const passwordValida = await bcrypt.compare(contrasena, usuario.contrasena);
if (!passwordValida) {
```

```
    return res.status(401).json({ mensaje: "Correo o contraseña incorrectos" });
}

const token = jwt.sign(
    {
        id_usuario: usuario.id_usuario,
        email: usuario.email,
        rol: usuario.rol
    },
    process.env.JWT_SECRET,
    { expiresIn: "8h" }
);
```

Recuperación de contraseña con OTP

```
const generarOTP = () => {
    return Math.floor(100000 + Math.random() * 900000).toString();
};

// recuperarPassword
const codigo = generarOTP();
const expiracion = new Date(Date.now() + 15 * 60 * 1000);
// ... INSERT INTO codigos_verificacion (id_usuario, codigo, expiracion)

// verificarCodigo
if (otp.usado) {
    return res.status(400).json({ mensaje: "El código ya fue utilizado" });
}
if (new Date() > new Date(otp.expiracion)) {
    return res.status(400).json({ mensaje: "El código expiró" });
```

```
}
// ... UPDATE codigos_verificacion SET usado = true WHERE id_verificacion = $1
const tokenRecuperacion = crypto.randomBytes(32).toString("hex");
// ... UPDATE codigos_verificacion SET token_recuperacion = $1 WHERE id_verificacion = $2
```

Crear una reserva

```
const client = await pool.connect();
try {
  await client.query('BEGIN');
  // ... valida usuario (rol USUARIO, tipo ESTUDIANTE o EGRESADO),
  //     laboratorio existente y DISPONIBLE, y fecha no pasada

  // Horario de funcionamiento del día de la semana
  const horarioResult = await client.query(`
    SELECT id_horario FROM horario
    WHERE dia_semana = EXTRACT(ISODOW FROM $1::date)
      AND $2::time >= hora_apertura
      AND $3::time <= hora_cierre`,
    [fecha, horaInicio, horaFin]);

  // Cruce con otra reserva del mismo usuario
  const reservaUsuarioResult = await client.query(`
    SELECT id_reserva FROM reserva
    WHERE id_usuario = $1 AND fecha = $2
      AND estado IN ('PENDIENTE', 'CONFIRMADA')
      AND hora_inicio < $4::time AND hora_fin > $3::time
    LIMIT 1`,
    [id_usuario, fecha, horaInicio, horaFin]);
```

```
    // Cupos del laboratorio en esa franja
    const disponibilidadResult = await client.query(`
        SELECT COUNT(*) AS reservas_actuales FROM reserva
        WHERE id_laboratorio = $1 AND fecha = $2
          AND estado IN ('PENDIENTE', 'CONFIRMADA')
          AND hora_inicio < $4::time AND hora_fin > $3::time`,
        [id_laboratorio, fecha, horaInicio, horaFin]);
    if (reservasActuales >= laboratorio.capacidad) {
        throw new Error('No hay cupos disponibles para el laboratorio en el horario seleccionado');
    }

    // INSERT INTO reserva (...) VALUES (..., 'CONFIRMADA')
    await client.query('COMMIT');
    return nuevaReserva.rows[0];
} catch (error) {
    await client.query('ROLLBACK');
    throw error;
} finally {
    client.release();
}
```

Asistencia

```
CREATE UNIQUE INDEX uq_asistencia_abierta
ON asistencia (id_usuario)
WHERE hora_salida IS NULL;
const registrarSalida = async (idUsuario) => {
    const consulta = `
```

```
      UPDATE asistencia
      SET hora_salida = CURRENT_TIMESTAMP
      WHERE id_usuario = $1
        AND hora_salida IS NULL
      RETURNING id_asistencia, id_usuario, id_laboratorio, ...`;
  const resultado = await pool.query(consulta, [idUsuario]);
  return resultado.rows[0] || null;
};
```

Contenedores

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
services:
 db:
   image: postgres:16-alpine
   container_name: saici-postgres
   environment:
     POSTGRES_DB: saici_db
     POSTGRES_USER: postgres
   ports: ["5432:5432"]
   volumes:
     - postgres_data:/var/lib/postgresql/data
     - ./BD_Saici.sql:/docker-entrypoint-initdb.d/init.sql:ro
```

```
  pgadmin:
    image: dpage/pgadmin4
    container_name: saici-pgadmin
    ports: ["5050:80"]
    depends_on: [db]
  backend:
    build: .
    container_name: saici-backend
    depends_on: [db]
    environment:
      PORT: 3000
      DB_HOST: db
      DB_PORT: 5432
      DB_NAME: saici_db
      DB_USER: postgres
    ports: ["3000:3000"]
volumes:
  postgres_data:
```

App Flutter

```
static String get baseUrl {
  if (kIsWeb) {
    return 'http://localhost:3000/api';
  }
  if (defaultTargetPlatform == TargetPlatform.android) {
    return 'http://10.0.2.2:3000/api';
  }
  return 'http://localhost:3000/api';
```

```
}

static Map<String, String> _headers({bool needsAuth = true}) {
  final headers = <String, String>{
    'Content-Type': 'application/json',
    'Accept': 'application/json',
  };
  if (needsAuth) {
    final token = AuthSessionService().token;
    if (token != null && token.isNotEmpty) {
      headers['Authorization'] = 'Bearer $token';
    }
  }
  return headers;
}
```

Crear una reserva desde la app

```
static Future<Map<String, dynamic>> createReserva({
  required int idLaboratorio,
  required String fecha,       // YYYY-MM-DD
  required String horaInicio,  // HH:MM
  required String horaFin,     // HH:MM
}) async {
  try {
    final response = await http.post(
      Uri.parse('$baseUrl/reservas'),
      headers: _headers(),
      body: jsonEncode({
```

```
        'id_laboratorio': idLaboratorio,
        'fecha': fecha,
        'horaInicio': horaInicio,
        'horaFin': horaFin,
      }),
    );
    if (response.statusCode == 201) {
      return {'success': true};
    } else {
      return {'success': false, 'error': _extractErrorMessage(response)};
    }
  } catch (e) {
    return {'success': false, 'error': 'Error de conexión al crear reserva.'};
  }
}
```

Sesión guardada

```
static const String _keyToken = 'saici_token';
static const String _keyUser = 'saici_user';

bool get isAuthenticated => _cachedToken != null && _cachedToken!.isNotEmpty;
String get currentRole => (_cachedUser?.rol ?? '').toUpperCase();

Future<void> saveSession(String token, UserModel user) async {
  // ...
  await prefs.setString(_keyToken, token);
  await prefs.setString(_keyUser, jsonEncode(user.toJson()));
}
```

Protección de rutas (lib/core/routes/app_routes.dart). Sin sesión se envía al login, y el rol decide qué pantalla de inicio se muestra.

```
if (!session.isAuthenticated) {
  return MaterialPageRoute(builder: (_) => const LoginScreen());
}

final rol = session.currentRole;
final tipo = session.currentTipo;
final isAdmin = rol == 'ADMINISTRADOR';
final isLab = rol == 'LABORATORIO';
final isEstudiante = rol == 'USUARIO' ||
    (!isAdmin && !isLab && (tipo == 'ESTUDIANTE' || tipo == 'EGRESADO'));

if (name == '/home') {
  if (isAdmin) {
    return MaterialPageRoute(builder: (_) => const AdminDashboardScreen());
  }
  if (isLab) {
    return MaterialPageRoute(builder: (_) => const LabDashboardScreen());
  }
  if (isEstudiante) {
    return MaterialPageRoute(builder: (_) => const UserHomeScreen());
  }
  // ... pantalla «Acceso Restringido» }
```

9. Privilegios

9.1 Roles y permisos

Rol	¿Cómo se identifica?	¿Qué puede hacer?
Sin sesión	—	Registrarse, iniciar sesión y recuperar la contraseña. En main también puede consultar y modificar laboratorios y horarios
Estudiante	rol USUARIO y tipo_usuario ESTUDIANTE	Explorar laboratorios, reservar, consultar y cancelar sus reservas, ver su historial y su perfil, y registrar su asistencia
Egresado	rol USUARIO y tipo_usuario EGRESADO	Lo mismo que el estudiante, salvo la asistencia, que el servidor reserva a estudiantes
Administrador	rol ADMINISTRADOR	Aprobar o rechazar reservas y gestionar laboratorios, horarios, reportes y usuarios (manual de usuario del informe)
Encargado de laboratorio	rol LABORATORIO (en BD_Saici.sql)	Cambiar el estado de las reservas y crear o editar laboratorios (rama feature/aplicativofuncional); en la app, su panel y sus reservas

9.2 Control de acceso en la API

Nivel	Rutas	¿Cómo se aplica?
Público	/, register, login, recuperar-password, verificar-codigo, nueva-password	Sin validación
Con token	/api/auth/perfil, cambiar-password, usuarios, /api/reservas	authMiddleware valida el Bearer JWT
Con token y estudiante	/api/asistencia/* (solo main)	estudianteMiddleware consulta tipo_usuario en la BD y responde 403 si no es ESTUDIANTE
Dueño de la reserva	Editar y cancelar reservas	El servicio verifica que la reserva sea del usuario autenticado

Nivel	Rutas	¿Cómo se aplica?
Administrador	GET /api/reservas	Si el rol es ADMINISTRADOR devuelve todas las reservas; a los demás, solo las suyas
Administrador o laboratorio	PATCH /api/reservas/:id/estado; POST y PUT de laboratorios (DELETE, solo administrador)	autorizarRoles en feature/aplicativofuncional
Sin validación	/api/laboratorios y /api/horarios en main	Ninguna